HKDSE Information and Communication Technology 香港中學文憑考試 資訊及通訊科技

SBA

香港道教聯合會圓玄學院第二中學

6B

陳鴻哲

目錄

Introduction: 4

A. 卡牌游戲簡介 4

Chapter 1: 程序界面 5

1.1 登錄畫面 5

1.2 注冊畫面 5

1.3 游戲畫面 6

Chapter 2: 用戶數據的處理 7

2.1 數據儲存設計和處理 7

2.2 數據檢驗 7

Chapter 3: 程序編寫: 9

3.1 程式總預覽 9

3.2 程式基礎設定 11

3.3 數據類型和數據結構 12

3.4 模塊化編寫 13

3.5 程序內數據交互 18

3.6 算法編寫 20

3.7 算法優化 24

Chapter 4: 程式運行測試: 25

4.1 組件單元測試.....25

4.2 登錄與注冊界面測試.....26

4.3 游戲界面測試.....29

Chapter 5: 錯誤處理: 30

5.1 主程序異常響應.....30

Conclusion: 31

A. 總結 31

B. 總代碼 32

Total 69

卡牌游戲簡介

本項目是基于 Python 编程語言開發的德州撲克游戏（Texas Hold'em）。通过设计和编码成功地实现了德州扑克这一经典纸牌游戏的数字化版本。玩家可以在圖像界面上体验到真实的德州扑克游戏乐趣，与计算机进行對弈，展现自己的牌技和策略。

项目特点：

1. 使用 Python 编程語言實現，簡潔高效
2. 包含德州撲克游戏的核心规则和逻辑，玩家能够体验真实的游戏过程
3. 提供了全面的用户交互界面，使游戏非常具互动性
4. 考虑了牌局随机性和策略性，增加了游戏的挑战性和乐趣性

技术亮点：

1. 使用面向对象编程（OOP）的思想，合理划分游戏中的各个对象和功能
2. 利用 Python 的数据结构和算法，实现了牌局的随机洗牌、发牌功能和評估牌組
3. 通过条件语句和循环结构，实现了游戏规则的逻辑判断和流程控制

德州扑克基本玩法：

1. **下盲注**：游戏开始前，首两名玩家必须分别下小盲注和大盲注
2. **发手牌**：每位玩家會先獲得两張手牌，这两张牌只有自己可以看到
3. **下注轮次**：第一轮下注开始，从第一个玩家开始，选择跟注、加注、弃牌或者过牌
4. **前三张公共牌**：接着发三张公共牌
5. **下注轮次**：第二轮下注开始，玩家下注方式和之前一樣
6. **第四张公牌**：下注结束后，发放第四张公共牌
7. **下注轮次**：第三轮下注开始，玩家下注方式和之前一樣
8. **第五张公共牌**：下注结束后，发放第五张公共牌，这是最后一张公共牌
9. **最终下注轮次**：第四轮下注开始，玩家可以进行最后一轮下注操作
10. **摊牌**：最后剩下的玩家将亮出手牌，与公共牌组成最好的五张牌组合以决定胜负
11. **决定胜负**：根据牌型大小和规则，确定获胜者，赢得底池中的筹码

1.程序界面

1.1 登錄畫面(默認畫面)



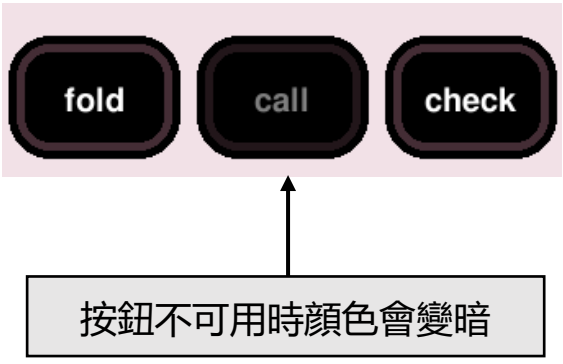
1.2 注冊畫面



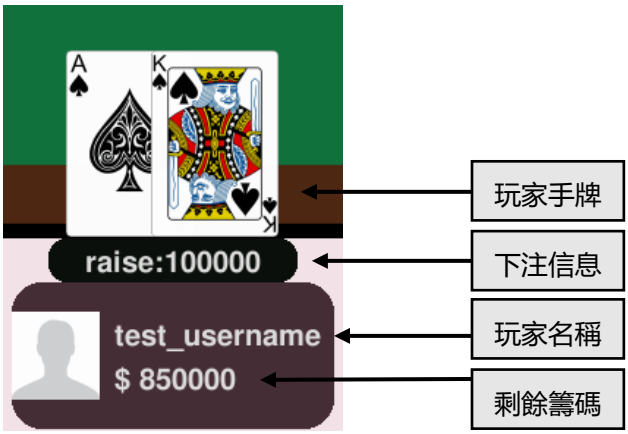
1.3 游戲畫面



下注按鈕顯示：

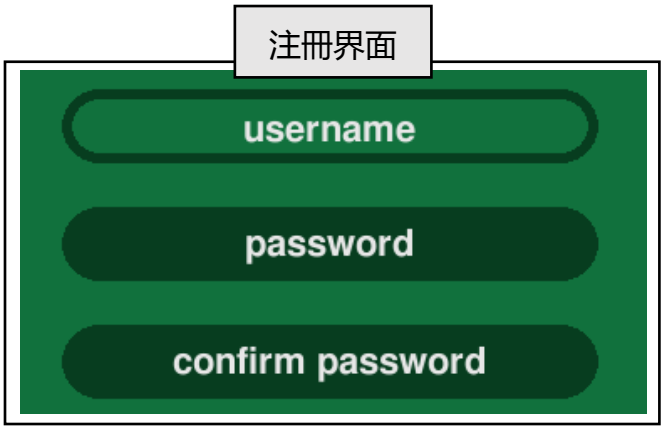


用戶信息顯示：



2.用戶數據的處理

2.1 用戶輸入的數據檢驗



注冊界面：

在注冊進程中，玩家需要為自己創建一個獨一無二的用戶名和长度必须为 8 位並包含数字和字母的密碼，游戲中提示為(password must be 8 characters long with numbers and letters)。

為確保用戶在(password)框内準確輸入新密碼，還需在其下方(confirm password)框内再輸入一次密碼，最終按下注冊按鈕后提交信息至後台驗證

後台驗證：

數據傳入後台后，將新用戶名與所有用戶名做比對，通過後則繼續校驗兩次密碼是否相同，再而用數據類型和長度檢查進行校驗。最終通過后會通知用戶注冊成功，游戲中提示為(sign up successful)。登錄則直接與數據庫中的用戶做匹配，成功后便顯示登陸成功，游戲中提示為(sign in successful)。

2.2 數據存儲設計

所有用戶數據：

文件名稱	player.json
目的	此文件包含用戶的屬性。用於登錄系統
數據層級關係	"player_data" 數組包含了所有用戶對象，每個對象的數據以獨立的字典方式存儲

用戶屬性：

屬性名	解釋	例子
type	用戶的類型，只有整數 0 和 1，分別是電腦和真實用戶	0
username	用戶名，類型為字符串	Peter
password	登陸密碼，類型為字符串	1234abcd
chip	用戶剩餘籌碼，類型為非負整數	100000

示例數據文件與格式：

```
{
    "player_data": [
        {
            "type": 0,
            "username": "Peter",
            "password": "832jce83912deddwa",
            "chip": 10000
        },
        {
            "type": 0,
            "username": "Amy",
            "password": "832jce83912deddwa",
            "chip": 0
        },
        {
            "type": 1,
            "username": "Tom",
            "password": "1234hello",
            "chip": 5000
        },
        {
            "type": 0,
            "username": "John",
            "password": "832jce83912deddwa",
            "chip": 50000000
        },
        {
            "type": 0,
            "username": "Alex",
            "password": "832jce83912deddwa",
            "chip": 5400000
        },
        {
            "type": 1,
            "username": "Sam",
            "password": "heyictsba2025",
            "chip": 1000000
        }
    ]
}
```

3.程序編寫

3.1 程式總預覽

程序文件夾結構:

```
├─code
│      base.py
│      button.py
│      evaluator.py
│      holdem.py
│      input_box.py
│      keywords.py
│      login.py
│      main.py
│      player.py
│      rendering.py
│      setting.py
│
├─data
│      player.json
│
├─font
│      FreeSansBold.ttf
│
├─image
    │  card_back.png
    │  User.jpg
    │
    ├─cards
    │      10_1.png
    │      10_2.png
    │      10_3.png
    │      10_4.png
    │      ...
    ├─chips
           black.png
           ...
```

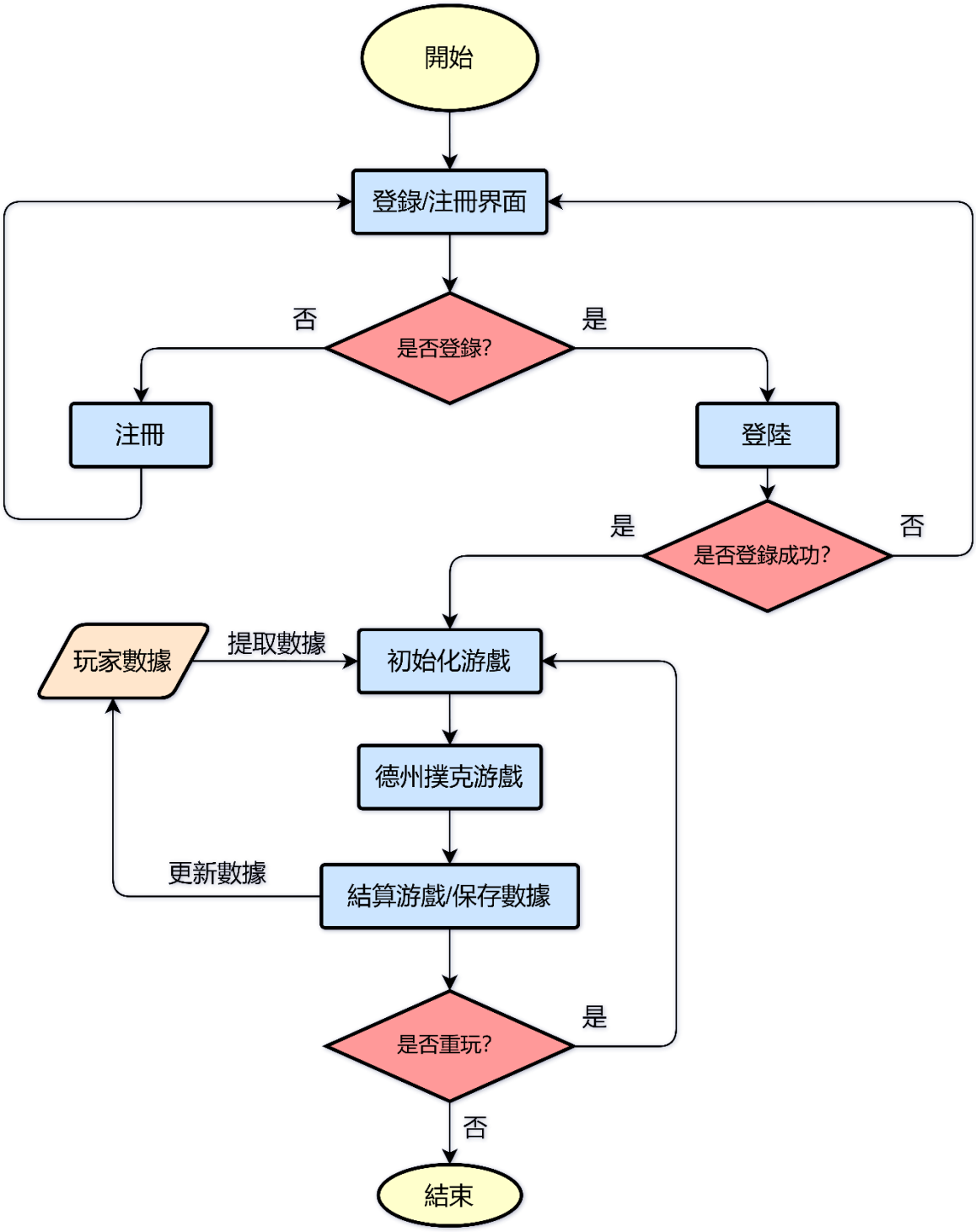
文件夾作用:

文件夾名稱	作用
code	程序原代碼
data	存儲用戶數據
Font	程序界面顯示的字體
image	程序內使用的圖片

code 文件夾:

文件名稱	作用
main.py	程序運行入口
setting.py	程序渲染配置文件
keywords.py	儲存程序中的關鍵字或字串
login.py	存儲用戶信息處理函數
base.py	存儲游戲的基類(base class)
holdem.py	德州撲克模塊
evaluator.py	計算玩家手牌組合等級
player.py	儲存玩家操作類
rendering.py	圖像渲染模塊
button.py	存儲按鈕的類
input_box.py	存儲輸入框的類

程式運行結構圖：


開始
登錄/注冊界面
否
是
是否登錄?
注冊
登陸
是
否
是否登錄成功?
玩家數據
提取數據
初始化游戲
德州撲克游戲
更新數據
結算游戲/保存數據
是
是否重玩?
否
結束

3.2 程式基礎設定

運行環境：

1. 程式僅支持 ASCII 編碼中字符作輸入，不支持的字符將呈現「」符號
2. 程式所需的解释器(interpreter)版本至少为 python 3.10(應用了新版本語句 match... case ...)
3. 程式依賴內置模塊 random、os、json、itertools 和第三方模塊 pygame
4. 安裝 pygame 需要在命令行(cmd)內輸入 pip install pygame（需要將 pip 添加至环境变量）

程式個性化設定：

1. 可在 setting.py 文件中更改程式中畫面的設定，如幀數、畫面長度、畫面寬度、顏色、位置等
2. 可在 keywords.py 文件中更改程式中顯示的文字內容，如的登陸提示、下注提示等

setting.py 文件示例：

```
# screen setting
SCREEN_HEIGHT= 750
SCREEN_WIDTH = 1380
SCREEN_COLOR = (242, 225, 231)

# FPS setting
FPS = 120

# title
TITLE_FONT_SIZE = 72
```

keywords.py 文件示例：

```
# button
ALL_IN = 'all in'
CHECK = 'check'
BET_RAISE = 'raise'
FOLD  = 'fold'
CALL = 'call'
INCREASE = '+'
DECREASE = '-'

#login info
SIGN_IN_INFO = 'Please login'
SIGN_IN_SUCCESSE = 'sign in successful'
```

3.3 數據類型和數據結構

數據類型：

程式中使用的數據類型有布爾值(bool)、整數(int)、字符串(str)、浮點數(float)、列表(list)、元組(tuple)、字典(dict) 與類(class)

類型	作用	例子
布爾值	儲存二元狀態	`CHEATING_MODE = False`
整數	存儲游戲基本整數數據	`FPS = 120`
字符串	存儲游戲內提示和字典的鍵名	`SIGN_IN_INFO = 'Please login'`
浮點數	存儲進行精确计算后的值，如坐標變換后的值	`POKER_HEIGHT = 127.05`
列表	存儲整合后的數據	`buttonList = [b1,b2,b3,...]`
元組	存儲畫面設置，如坐標，RGB 顏色	`BUTTON_COLOR = (0,0,0)`
字典	儲存键值对数据，如圖片名所對應的圖片對象	`{file_name:image_surface,...}`
類	自定義數據，以創建游戲規則	`class BaseGame:...`

數據結構：

程式使用的數據結構有二維數組(two-dimensional array)，和優先隊列(priority queue)，其餘則繼續使用列表(list)和元組(tuple)

類型	原因	例子
二維數組	有直观的嵌套數據表示方式，方便提取數據	`HAND_POSITION = [[pos_1,pos_2],...]`
優先隊列	能夠直接獲取優先度最高的數據，e.g. 以玩家手牌等級作為優先度，來獲得做高優先玩家，即勝家	`winnersList = PriorityQueue()`

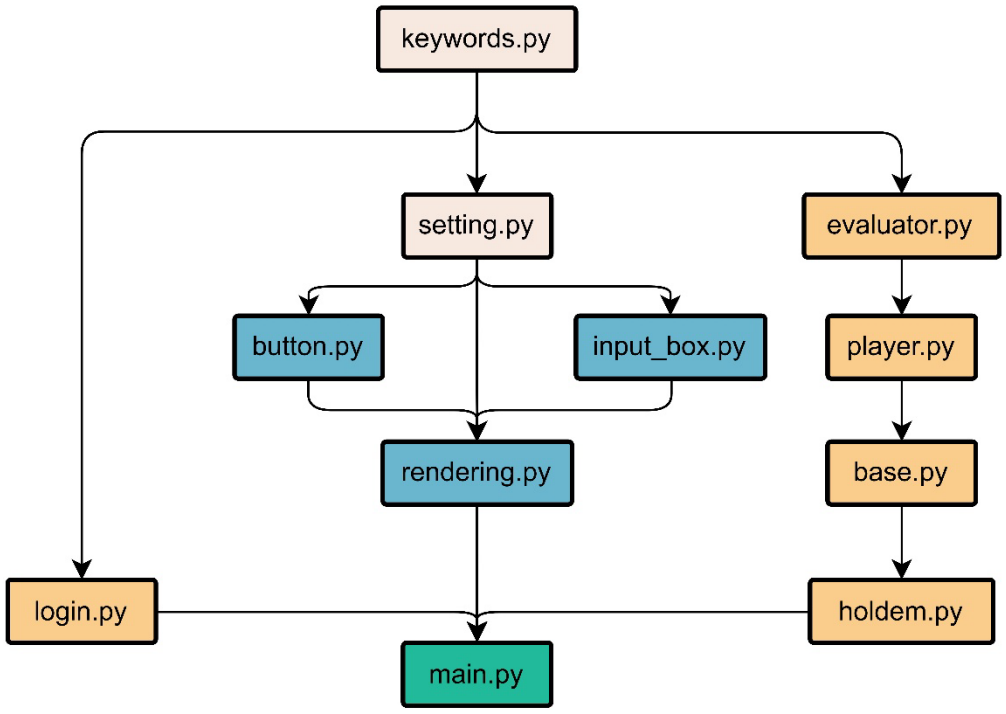
3.4 模塊化編寫

程式大致分爲四大模塊——配置文件、渲染組件、計算組件、主程序：

1. 配置文件：管理所有可配置變量
2. 渲染組件：接收計算組件傳入的數據以渲染給玩家
3. 計算組件：計算提交給程式數據，並返回信息，沒有任何畫面反饋
4. 主程序：將渲染組件和游戲組件合并以運行游戲

配置文件	渲染組件	計算組件	主程序
setting.py	rendering.py	login.py	main.py
keywords.py	button.py	base.py	
	input_box.py	holdem.py	
		player.py	
		evalutor.py	

模塊關係圖：



配置文件:

1. setting.py——特殊定義

定義	作用
`class QuitGame(Exception):...`	繼承錯誤基類(Exception)，調用此類將觸發異常，安全退出游戲
`poker_id`	e.g(14_4)前部以 2-14 代表 2-A 的撲克點數,后部 1-4 代表花色
`imageList`	存儲所有卡牌的字典{卡牌 id:圖片對象,...}
`seat`	代表為玩家的座位索引，範圍是 0-4，以最左側為 0 到最右側 4

渲染組件:

1. Button.py——存儲 Button 類,內置 ***3*** 個方法(method),創建按鈕實例(instance) 以供用戶點擊按鈕

Button 類

方法	作用
`def adjust_value():...`	調整按鈕隱藏數值，即可能自定義點擊按鈕后自增 1
`def check():...`	檢查按鈕是否被點擊
`def draw():...`	渲染按鈕至用戶界面

2. Input_box.py——存儲 Input 類,內置 ***3*** 種方法,創建輸入框實例以供用戶輸入信息

Input 類

方法	作用
`def content_init():...`	重置輸入框內容
`def check():...`	檢查輸入框是否被輸入
`def draw():...`	渲染輸入框至用戶界面

3. rendering.py——內置 ***2*** 個預渲染函數，***15*** 個渲染函數

預渲染函數(調用將返回圖形對象，一般供渲染函數使用)

函數	作用
`def render_reminder():`	返回包含提示語的圖形對象
`def render_pot_info():`	返回獎金池圖形對象

渲染函數(調用即渲染在用戶界面)

函數	作用
`def draw_ranking():...`	在登陸/注冊界面渲染用戶籌碼排行榜
`def draw_introduction():...`	在登陸/注冊界面渲染入場動畫
`def draw_temp_reminder():...`	渲染短暫提示
`def draw_table():...`	渲染賭桌在游戲界面
`def draw_players():...`	渲染玩家用戶名和剩餘籌碼在游戲界面
`def draw_combo():...`	渲染玩家牌組名稱(e.g. One Pair,Flush)在游戲界面
`def draw_pot():...`	渲染游戲總獎金在游戲界面
`def draw_betting():...`	渲染玩家最新的下注信息(e.g. call 3000,check)在游戲界面
`def draw_winner():...`	渲染獲勝玩家用戶名和牌組名稱在游戲界面
`def draw_card():`	渲染單張卡牌
`def draw_hand():...`	渲染單個玩家的手牌在游戲界面
`def draw_community():...`	渲染公牌在游戲界面
`def draw_consideration():...`	渲染進度條在游戲界面，以模擬電腦思考時間
`def move_animation():...`	傳入渲染對象和起始和終點坐標，對象將按直綫軌跡運動
`def interact():`	用戶與程式交互的接口，將渲染按鈕和輸入框

計算組件:

1. login.py——內置 ***7*** 個函數

函數	作用
`def load_players():...`	返回所有用戶的賬號數據
`def sign_in():...`	傳入賬號密碼以登錄，成功后將返回該賬號的數據，否則 False
`def sign_up():...`	傳入賬號密碼以創建賬號，成功后將返回 True，否則 False
`def get_testing_data():...`	返回測試賬號數據，其數據為臨時創建，不會作保存
`def get_bot():...`	返回機器人的賬號數據
`def save_game():...`	傳入新數據以更新用戶數據
`def calculate_ranking():...`	返回根據籌碼排名的用戶列表

2. base.py——存儲 BaseGame 類，內置 ***5*** 個方法，所有游戲將繼承該類作基本處理

BaseGame 類

方法	作用
`def init_player():...`	初始化包含玩家對象的列表
`def save_game():...`	返回當前游戲的全部玩家數據
`def get_players_info():...`	返回全部玩家的單一屬性，e.g. 按 seat 返回所有玩家的用戶名
`def playersList():...`	返回包含玩家對象的列表，未初始化時列表為空
`def game_loop():...`	游戲回合的抽象方法

3. holdem.py——存儲 Holdem 類，內置 ***6*** 個方法，繼承 BaseGame 類

Holdem 類

方法	作用
`def check_game():...`	檢查游戲初始化是否正確
`def game_loop():...`	調用將運行單個游戲回合
`def betting_round():...`	單個回合中的下注細節運算
`def check_winner():...`	將勝家資訊儲存在 self.winnerList 列表中
`def deal_player():...`	發放手牌給玩家對象和二維數組 self.handList
`def deal_community():...`	發放公派給 self.communityCardsList 列表

4. player.py——存儲 Player 和 Bot 類，共 ***3*** 個方法，Bot 繼承 Player 類，供玩家進行操作

Player 類

方法	作用
`def decision():...`	以字典形式返回玩家對游戲回合的操作
`def combination():...`	評估為玩家的牌組等級，並存儲在 self.combo 中

Bot 類

方法	作用
`def decision():...`	重寫了真人操作部分，以隨機數做操作代表機器人，返回值一樣

5. evaluator.py ——存儲 Card、LookupTable 和 Evaluator 類，共 ***19*** 個方法(細節見 *3.6 算法設計*)

Card 類

方法	作用
`def new():...`	傳入卡牌 id 以返回卡牌的唯一標識碼
`def int_to_str():...`	傳入卡牌的唯一標識碼以返回卡牌 id
`def get_rank_int():...`	傳入卡牌的唯一標識碼以返回 0 為始的卡牌等級
`def get_suit_int():...`	傳入卡牌的唯一標識碼以返回花色數值
`def get_bitrank_int():...`	傳入卡牌的唯一標識碼以返回卡牌等級
`def get_prime():...`	傳入卡牌的唯一標識碼以返回卡牌對應的質數
`def hand_to_binary():...`	傳入卡牌的唯一標識碼以返回二進制標識碼
`def prime_product_from_hand():...`	傳入卡牌列表以通過卡牌質數計算牌組數值
`def prime_product_from_rankbits():...`	傳入卡牌列表以通過卡牌等級計算牌組數值

LookupTable 類

方法	作用
`def flushes():...`	更新全為同花的查找表
`def straight_and_highcards():...`	更新非同花的查找表
`def multiples():...`	更新非同花的查找表
`def get_lexographically_next_bit_sequence():...`	按字典顺序计算下一个位排列

Evaluator 類

方法	作用
`def evaluate():...`	傳入牌組以返回牌組編號
`def _two():...`	計算兩張牌時的牌組編號
`def _five():...`	計算五張牌時的牌組編號
`def _six():...`	計算六張牌時的牌組編號
`def _seven():...`	計算七張牌時的牌組編號
`def get_rank_string(): ...`	傳入牌組編號以返回牌組等級名稱

主程序：

1. main.py——內置 ***3*** 個函數

函數	作用
`def login_page():`	登錄/注冊界面
`def holdem_page():`	德州撲克游戲界面
`def main():`	程式入口

3.5 程序內數據交互

實時渲染：

在 main.py 內，holdem.py 與 rendering.py，login.py 與 rendering.py 之間的函數或方法互相傳參，讓用戶界面的操作反饋到計算組件，計算後再返回數據讓渲染組件更新畫面

Login_page 函數示例

代碼	解釋
`draw_ranking(calculate_ranking(load_players()))`	1. 將 load_players()返回的玩家數據傳給 calculate_ranking() 2. calculate_ranking()將返回名次排序 3. 將名次排序傳入 draw_ranking()，進而渲染到用戶界面

`result = interact(...)`
`if result == SIGN_IN:`
`  ...`
`  login = sign_in(result)`
`  if login:`
`  draw_temp_reminder()`
`else:...`
	1. Interact()返回的用戶操作存入 result
	2. 判斷是否登錄
	3. 將用戶結果(輸入的賬號密碼)傳入 sign_in()驗證
	4. 登錄成功將渲染成功提示

holdem_page 函數的數據交互示例：

holdem_page 函數中的用戶界面交互較爲複雜，以高階函數(Higher-order function)為基礎編寫了游戲主循環，原因在于單個下注循環中需要實時渲染玩家的下注操作，必須接收輸入類函數以做到同時計算和接收反饋，輸出類函數則是可選的(默認為無輸出)

代碼	解釋

`game.game_loop(interact,`
`draw_players,`
`draw_hand,`
`draw_betting,`
`draw_consideration)`
	游戲主循環 game_loop()
	1. Interact (用戶操作反饋)，
	2. draw_players(渲染玩家實時籌碼等信息)
	3. draw_hand(渲染玩家手牌，e.g.是否棄牌)
	4. draw_betting(渲染玩家最新的下注信息)
	5. draw_consideration(模擬電腦思考時間)

內部調用：

當渲染函數傳入 game_loop 后將繼續傳給有需要調用的方法，包括中的 Holdem 類的方法 betting_round 和 Player 類的方法 decision

Holdem.betting_round 方法示例(輸出)：

代碼	解釋

`if updateChipFunc:`
`updateChipFunc(...)`
`if updatePotFunc :`
`updatePotFunc(...)`
`if updateCardFunc and ...:`
`updateCardFunc(...)`
	1. 條件判斷是否傳入了輸出函數
	2. 輸出格式需參考文檔字符串(docString)
	3. 輸出函數可以以任何方式輸出，需按照輸出格式處理數據

Player.decision 方法示例(輸入)：

代碼	解釋
`result:dict = choiceFunc(...)`	1. 輸入函數指定了輸出類型為字典 2. 輸入函數將被傳入一些限制條件以限制用戶某些操作 3. 輸入函數可以以任何方式接收反饋，只需遵守返回值類型

3.6 算法編寫

算法簡介:

本程式除去基本德州撲克的回合算法，渲染圖像算法外，evaluator.py——存儲用以演算玩家手牌組合等級的算法，其算法概念為 Cactus Kev's Poker Hand Evaluator，實際代碼為第三方開源模塊 treys (可通過 pip 下載)，但在本程式中已經對此模塊中的代碼進行更改以迎合一些額外的對局規則

Cactus Kev's Poker Hand Evaluator 算法:

該算法將玩家的牌組等級分化成 1 至 7462 個編號，但顯然玩家能獲得的 5 張牌組合數為 C(52,5)，即 2,598,960 個組合，遠遠超過 7462。在這約 260 萬個組合中包含了花色的組合，例如梅花 A 和方塊 A，但他們的等級在德州撲克規則裏的等級是一樣的(不看花色)，所以只看如何平局以分勝負的組合可以是四人持有 Q 高同花順(四種花色) 、四人持有 J 高同花順、四人持有 10 高同花順，雖然花色不同，但等級一樣導致最終平局。將一種同花順化爲一個編號，從關注花色的 40 種同花順減少至共 10 種同花順 (A 至 10)

組合數表格

組合名	唯一的編號數量	分類后的編號數量
Straight Flush(同花順)	40	10
Four of a Kind(四條)	624	156
Full Houses(骷髏)	3744	156
Flush(同花)	5108	1277
Straight(順子)	10200	10
Three of a Kind(三條)	54912	858
Two Pair(兩對)	123552	858
One Pair(一對)	1098240	2860
High Card(高牌)	1302540	1277
總和	2598960	7462

各組合編號的枚舉：

同花順	同花	兩對
1：4 人持有 A 高同花順	323：4 人持有 AKQJ9 同花	2468：144 人持有 AAKKQ
2：4 人持有 K 高同花順	324：4 人持有 AKQJ8 同花	2469：144 人持有 AAKKJ
…	…	…
9：4 人持有 6 高同花順	1598：4 人持有 76432 张同花	3324：144 人持有 33225
10：4 人持有 5 高同花順	1599：4 人持有 75432 张同花	3325：144 人持有 33224
四條	**順子**	**一對**
11：4 人持四张 A 和一张 K	1600：1,020 人持有 A 高順子	3326：384 人持有 AAKQJ
12：4 人持四张 A 和一张 Q	1601：1,020 人持有 K 高順子	3327：384 人持有 AAKQT
…	…	…
165：4 人持有四张 2 和一张 4	1608 ： 1,020 人持 6 高順子	6184：384 人持有 22643
166：4 人持有四张 2 和一张 3	1609 ： 1,020 人持 5 高順子	6185：384 人持有 22543
骷髏	**三條**	**高牌**
167：24 人持有三張 A 和兩張 K	1610：64 人持有 AAAKQ	6186：1,020 人持有 AKQJ9
168：24 人持有三張 A 和兩張 Q	1611：64 人持有 AAAKJ	6187：1,020 人持有 AKQJ8
…	…	…
321：24 人持有三張 2 和兩張 4	2466：64 人持有 22253	7461：1,020 人持有 76432
322：24 人持有三張 2 和兩張 3	2467：64 人持有 22243	7462：1,020 人持有 75432

生成點數編號：

需要將一組牌，不管排序的情況下(A,2,3,4,5 或 2,4,A,5,3)換算成對應的組合編號運用了質數乘積的唯一性，算數基本定理(The fundamental theorem of arithmentic)，將每張卡牌點數都分配一個質數，這樣無論是什麼牌組都會獲得一個唯一的數值(類似 hash 值)，進而可以將該數值進行質因數分解以重新獲得到每張牌的點數

分配質數表

點數	2	3	4	5	6	7	8	9	10	J	Q	K	A
質數	2	3	5	7	11	13	17	19	23	29	31	37	41

唯一識別碼：

爲了讓花色也有識別碼，並和的點數合并，算法以 4 個字節的二進制數值創建了一套卡牌存儲方案

二進制數據(x 為禁用)

xxxAKQJT98765432	CDHS	rrrr	xxppppppp

1. 紅色格子：牌的等级,根据牌的花色將對應的位調為 1
2. 黃色格子：牌的花色(梅花=C,方塊=D,紅心=H ,黑桃=S)根据牌的花色將對應的位調為 1
3. 藍色格子：r = 以 0 為始的卡牌等级(點數 2=0,點數 3=1,點數 4=2,點數 5=3,...,點數 A=12)
4. 綠色格子：p = 等级的質數 (點數 2=2,點數 3=3,點數 K=37,...,點數 A=41)

卡牌編號示例

	xxxAKQJT 98765432	*CDHS*	*rrrr*	*xxppppppp*
方块 K	00001000 00000000	0100	1011	00100101
黑桃 5	00000000 00001000	0001	0011	00000111
梅花 J	00000010 00000000	1000	1001	00011101

計算組合：

算法將同花與同花順歸爲一類先進行檢測，因爲只檢測點數編號會忽略同花的組合，檢測方法便是利用計算出的二進制卡牌數值進行位元運算(Bitwise operation)，將 c1 至 c5 標記為 5 張手牌，如果得出為 0 則代表沒有同花，反之非零則有同花：

```
c1 & c2 & c3 & c4 & c5 & 0xF000
```

如有同花將繼續檢測是否為順子，繼續使用位元運算：

```
q = (c1 | c2 | c3 | c4 | c5) >> 16
```

這樣，便有了最低為 31(0b0001 1111),最高為 7936(0b1 1110 0000 0000),程式中 `LookupTable` 類的 `flushes` 方法裏創建了長度為 10 的 `straight_flushes` 列表，儲存了全部同花順的 q 值，只要將手牌的 q 值計算出再與該列表作遍歷查找便可得出是否為同花順，整體時間複雜度僅爲 O(n)

雖然有 straight_flushes 列表能直接進行查找，但對於其他同花或其他非同花組合需要創建兩個查找表(lookuptable)結構的字典，因此用循環創建了以 q 值為鍵，組合編號為值的 flush_lookup 和 unsuited_lookup 分別儲存有同花的組合和沒同花的組合，所以普通同花只需訪問查找表便能直接得到組合編號，例如 flush_lookup[7808]會返回 323(*AKQJ9 同花的組合編號*)，從而無需使用列表查找

當確定了牌組為非同花后，便能直接使用點數編號繼續計算了，對於這些手牌只需將 5 張牌的質數相乘:

```
q = (c1 & 0xFF)*(c2 & 0xFF)*(c3 & 0xFF)*(c4 & 0xFF)*(c5 & 0xFF)
```

得出 q 后便能直接訪問 unsuited_lookup 字典，獲得組合編號，時間複雜度為 O(1)，程式在犧牲空間的情況下擁有很快的執行效率

創建查找表的時間:

在創建 Evaluator 類的實例時會開始創建兩個查找表，雖然執行時間會較長(4.5-5ms)但這只會運行一次，兩個查找表便會一直在內存中直到程序關閉，只要直接用計算後的牌組數值去訪問查找表便能直接得到牌組編號，所以整體效率將比每獲得一張新牌就計算該牌組的等級快得多

使用該演算法:

在主程序運行時將會自動在 evalutor.py 內創建一個 Evaluator 的實例供使用，需要配和 Card 類以產生的單個卡牌的唯一識別碼

代碼示例:

代碼	解釋
`evaluator = Evaluator()`	創建實例
`hand = [Card.new('2_1'),Card.new('2_2')]`	通過卡牌 id 創建有為唯一標識碼的手牌列表
`result = evaluator.evaluate(hand,[])`	進行演算，將返回牌組編號

3.7 算法優化

舊算法:

在使用 Cactus Kev's Poker Hand Evaluator 前演算牌組的算法是每有新的牌組更新等都需要約 1ms 的時間進行演算，其中大部分函數的時間複雜度都為 O(n^2)，雖然在正常游戲中只會處理長度 2 至 7 的列表，優化效果并不明顯，但如果在未來有新卡牌游戲並且要處理更長的卡牌列表，這些算法所需的時間將會以平方的速度增加

檢測順子的函數(已棄用)示例

```
def straight(cards_rank_list):
    power = []
    ranks = list(set(cards_rank_list))
    length= 5                          # detect 5 consecutive number, length = 5
    if {2,3,4,5,14}.issubset(ranks): # special combination A,2,3,4,5
        power.append([1,2,3,4,5])
    for i in range(len(ranks) - length + 1):
        tempList = ranks[i:i + length]
        if all((tempList[j] + 1 == tempList[j + 1]) for j in range(length - 1)):
            power.append(tempList)
    return max(power) if power else power
```

檢測勝家算法的優化:

在 Holdem 類中的 check_winner 方法中運用了優先隊列的數據結構來尋找勝家，訪問最高優先級數據時的時間複雜度僅為 O(1)，比起普通列表遍歷查找或二分查找的時間複雜度至少也要 O(log n)，減少循環代碼次數的同時增加了代碼的可讀性

check_winner 方法示例

```
def check_winner(self):
    nonFoldList = PriorityQueue()
    # 獲得未棄牌的玩家信息
    for player in self.playersList:
        if not player.fold:
            nonFoldList.put((player.combo[0],player,player.combo[1]))
    # 獲勝玩家
    self.winnerList.append(nonFoldList.get())
```

4.程序運行測試

4.1 組件單元測試

測試流程:

將會選擇一部分關鍵的函數或方法進行測試，將輸出結果和預期效果做比對查看是否通過測試

Holdem 類:

測試代碼	預期輸出	實際輸出

`game = Holdme()`
`game.deal_community(range(5))`
`print(game.communityCardsList)`
	包含 5 個卡牌 id 的列表	['3_4', '4_3', '6_2', '11_1', '6_1']

login.py 內的函數:

測試代碼	預期輸出	實際輸出

`print(`
`sign_up('rex','1234abcd','1234abcd')`
`)`
	True	True

`print(`
`sign_in('rex','1234abcd')`
`)`
	該用戶的賬號信息
	(除了密碼)
		{'type': 1, 'username': 'rex', 'chip': 500000}

evaluator.py 內的方法:

測試代碼	預期輸出	實際輸出

`evaluator = Evaluator()`
`hand = [Card.new('10_1'),`
`Card.new('11_1')]`
`board = [Card.new('12_1'),`
`Card.new('13_1'),`
`Card.new('14_1')]`
`rank = evaluator.evaluate(hand,board)`
`print(evaluator.get_rank_string(rank))`
	Royal Flush	Royal Flush

4.2 登錄與注冊界面測試

測試流程：

將使用邊界案例(boundary case)進行測試，使用三个值，一个恰好在边界上，另外兩个分別在边界的两侧，將輸出結果和預期效果做比對查看是否通過測試，其次則將尋找其他同學進行測試

同學對此游戲的測試結果：

無任何報錯發生，且用戶界面引導良好

注冊界面測試：

- 將輸入用戶名，兩次相同的密碼
- 對用戶名無任何排列要求，但確保是唯一的
- 密碼最低要求必須為 8 位，並包含字母和數字

已知信息：

- 已注冊的用戶名有(*Peter，Amy，Tom，John，Alex，Sam*)

第一組(用戶名是否存在)

	輸入測試	預期輸出	實際輸出
用戶名	John	User exist or password invalid	User exist or password invalid
密碼	1234abcd		
密碼	1234abcd		
	輸入測試	預期輸出	實際輸出
用戶名	Texas	sign up successful	sign up successful
密碼	Texas1234		
密碼	Texas1234		

第二組(兩次密碼是否相同)

	輸入測試	預期輸出	實際輸出
用戶名	Oscar	User exist or password invalid	User exist or password invalid
密碼	Oscar1234		
密碼	Oscar123		

	輸入測試	預期輸出	實際輸出
用戶名	Oscar	sign up successful	sign up successful
密碼	Oscar1234		
密碼	Oscar1234		

第三組(密碼長度)

	輸入測試	預期輸出	實際輸出
用戶名	Jason	User exist or password invalid	User exist or password invalid
密碼	1234abc		
密碼	1234abc		
	輸入測試	預期輸出	實際輸出
用戶名	Jason	sign up successful	sign up successful
密碼	1234abcd		
密碼	1234abcd		
	輸入測試	預期輸出	實際輸出
用戶名	Jason	sign up successful	sign up successful
密碼	1234abcde		
密碼	1234abcde		

第四組(密碼是否包含字母)

	輸入測試	預期輸出	實際輸出
用戶名	Sunday	User exist or password invalid	User exist or password invalid
密碼	12345678		
密碼	12345678		
	輸入測試	預期輸出	實際輸出
用戶名	Sunday	sign up successful	sign up successful
密碼	12345678a		
密碼	12345678a		

第五組(密碼是否包含數字)

	輸入測試	預期輸出	實際輸出
用戶名	Felix	User exist or password invalid	User exist or password invalid
密碼	abcdefgh		
密碼	abcdefgh		
	輸入測試	預期輸出	實際輸出
用戶名	Felix	sign up successful	sign up successful
密碼	abcdefgh1		
密碼	abcdefgh1		

登錄界面測試:

- 將輸入用戶名和密碼

已知信息:

- 已注冊的用戶信息(用戶名: *rex* ， 密碼: *1234abcd*)

第一組

	輸入測試	預期輸出	實際輸出
用戶名	rex	sign up successful	sign up successful
密碼	1234abcd		
密碼	1234abcd		
	輸入測試	預期輸出	實際輸出
用戶名	Rex	User exist or password invalid	User exist or password invalid
密碼	1234abcd		
密碼	1234abcd		
	輸入測試	預期輸出	實際輸出
用戶名	rex	User exist or password invalid	User exist or password invalid
密碼	1234abcde		
密碼	1234abcde		

4.3 游戲界面測試

測試流程：

游戲界面的輸入方法只有按鈕(渲染組件)，因此只需關注按鈕能否在某些條件下設置為無效，例如有玩家加注后，check(過牌)按鈕將會被設置為無效，其次則將尋找其他同學進行測試

同學對此游戲的測試結果：

無任何報錯發生，且用戶界面引導良好

按鈕關鍵字列表(keywords.py 內)：

- CHECK(過牌)
- BET_RAISE(加注)
- FOLD(棄牌)
- CALL(跟住)
- INCREASE(增加注碼)
- DECREASE(減少注碼)

游戲界面按鈕測試

狀態	條件	預期渲染輸出	實際渲染輸出
玩家為盲注位	玩家只能下固定注碼	只有 call 按鈕有效	只有 call 按鈕有效
盲注回合但不是盲注位	玩家只能過牌	只有 check 按鈕有效	只有 check 按鈕有效
有其他玩家加注	玩家不能過牌	只有 check 按鈕無效	只有 check 按鈕無效
沒任何玩家加注	玩家不能跟住	只有 call 按鈕無效	只有 call 按鈕無效
玩家未獲得手牌	玩家不能棄牌	只有 fold 按鈕無效	只有 fold 按鈕無效
玩家只能全押(all in)	玩家不能加注或過牌	check，raise 按鈕無效	check，raise 按鈕無效

5.錯誤處理

5.1 主程序異常響應

處理異常:

程序在執行期間正常不會發生報錯現象，但發生概率也不為 0，所以編寫了一些緊急措施用作處理運行時的報錯，讓用戶能繼續使用程式，而不是突然關閉

主程序入口:

在 main 函數中編寫了 try 語句，用來放置可能會引發異常的的代碼，except 后寫了 QuitGame，用來檢測是否關閉游戲，而 Exception 則用作接收除 QuitGame 外的其他異常類(Exceptions)。如果觸發了報錯現象則重新調用 main 函數自身，回到登錄頁面，從而避免關閉程序

代碼

```
def main():
    try:
        login_page()
        holdem_page()
    except QuitGame:
        ...
        quit()
    except Exception:
        main()
```

總結

面向对象设计

面向对象编程的引入使得程序的各个组成部分能够更好地进行模块化。每个类负责不同的功能。例如，创建了 Player 类来管理玩家信息，Holdem 类来处理牌组的生成和洗牌。这种设计使得代码更易于理解和扩展，新的功能可以通过添加新类或方法来实现，而不需要对现有代码进行大幅修改。

登录系统

为了提升游戏的互动性和安全性，实现了一个用户登录系统。用户可以创建账户并登录，这样可以保存其游戏记录和统计数据。此系统使用简单的文件存储机制，确保用户数据的安全性和隐私。在用户登录时，系统会验证用户名和密码，并提供相应的反馈

德州扑克规则实现

游戏的核心是德州扑克的规则实现。我编写了一系列函数来处理发牌、下注、比牌等操作。这些函数的设计遵循单一职责原则，每个函数只处理特定的任务，从而提高了代码的可读性和可测试性。

Cactus Kev's Poker Hand Evaluator

为了准确评估手牌，集成了 Cactus Kev's Poker Hand Evaluator 算法。这一算法能够高效地判断不同手牌的强度，通过使用这个算法能够在游戏中快速且准确地比较玩家的手牌，提高了游戏的性能

错误处理机制

在开发过程中，特别是在处理用户输入和游戏逻辑时。我实现了全面的错误处理机制，确保在发生异常时，程序能够优雅地恢复，而不是崩溃。通过使用 try 和 except 块，我能够捕获常见的错误，如输入格式错误、运行时异常等

结论

通过这个项目，我不仅提升了自己的 Python 编程能力，还深入理解了面向对象编程的原则和实践。德州扑克游戏的开发过程让我体会到设计模式的重要性，以及如何将复杂的逻辑分解为可管理的组件。未来，我计划继续优化游戏的功能，增加更多的互动元素和图形效果，为玩家提供更加丰富的游戏体验。这个项目不仅是一个有趣的实践，更是我学习和成长的一个重要里程

總代碼

base.py:

```
import random
from player import *

class BaseGame:
    """game framework"""
    def __init__(self,total_player:int) -> None:
        #playerslist: all player information(username,chip)
        self.__playersList:list[Player] = []
        self.total_player:int = total_player

    @property
    def playersList(self):
        '''for child class, cheat mode'''
        return self.__playersList

    def save_game(self) -> list[dict]:
        '''return latest player info'''
        return [{kw.USERNAME:player.username,kw.CHIP:player.chip} for player in
self.__playersList]

    def get_players_info(self,attr:str) -> list:
        '''for draw func\n
        return a list include all player info\n
        find the attr in Player class'''
        return [getattr(player,attr) for player in self.__playersList]

    def init_player(self,bot_list:list,real_player:dict = {}):
        random.shuffle(bot_list)
        for bot in bot_list:
            self.__playersList.append(Bot(bot[kw.USERNAME],bot[kw.CHIP]))
        if real_player:
            self.__playersList.insert(2,(Player(real_player[kw.USERNAME],real_player[kw
.CHIP])))
        self.__playersList = self.__playersList[:self.total_player]
        # if len(self.__playersList) < self.total_player:
        #     raise 'init error'

    def game_loop(self):...

if __name__ == '__main__':
    pass
```

button.py:

```
from setting import *

class Button:
    '''draw a button in a Surface'''
    def __init__(self,
                 text:str,
                 rect:pygame.Rect,
                 color:tuple,
                 touch_color:tuple,
                 active_color:tuple,
                 active_touch_color:tuple,
                 flag:int = 0):
        """flag: function type of button,setting == -1 betting == 0,bet setting == 1"""
        self.text  = text
        self.rect  = rect
        self.color = color
        self.touch_color = touch_color
        self.active_color = active_color
        self.active_touch_color = active_touch_color
        self.default_color = color
        self.default_touch_color = touch_color
        self.flag  = flag

    def adjust_value(self,value):
        # increase /decrese
        if   self.text == kw.INCREASE: return  value
        elif self.text == kw.DECREASE: return -value

    def check(self, event:pygame.event):
        """return True when the button release"""
        # reset color in every loop
        self.default_color = self.color
        self.default_touch_color = self.touch_color
        if self.rect.collidepoint(pygame.mouse.get_pos()):
            self.default_touch_color = self.active_touch_color
            # press
            if event.type == pygame.MOUSEBUTTONDOWN:
                self.default_color = self.active_color
            # release
            elif event.type == pygame.MOUSEBUTTONUP:
                self.default_color = self.color
                self.default_touch_color = self.touch_color
                return True
        return False
```

```
    def draw(self, invalid:bool,value = None):
        """draw the button\n
           value will render above the button text"""
        pygame.draw.rect(screen,self.default_color, self.rect,0,25)
        pygame.draw.rect(screen,self.default_touch_color, self.rect,10,25)
        pygame.draw.rect(screen,self.color, self.rect,5,25)
        text = BUTTON_FONT.render(self.text,True,BUTTON_FONT_COLOR)
        screen.blit(text,((self.rect.x + self.rect.width/2) -
text.get_width()/2,(self.rect.y + self.rect.height/2) - text.get_height()/2))
        if value:
            value = BETTING_SIZE_FONT.render(str(value),True,BUTTON_FONT_COLOR)
            screen.blit(value,(self.rect.x + (self.rect.width -
value.get_width())/2,(self.rect.y + self.rect.height/2) - text.get_height() - 5))
        if invalid:
            invalid_layer =
pygame.Surface((self.rect.width,self.rect.height),pygame.SRCALPHA)
            pygame.draw.rect(invalid_layer,(0,0,0,128),invalid_layer.get_rect(),0,25)
            screen.blit(invalid_layer,self.rect)
        pygame.display.flip()

## =====NO CHANGE=====

# bet button
__betButton_width = 105
__betButton_height= 75
__betButton_y = PLAYER_INFO_BAR_POSITION[2][1] + (PLAYER_INFO_BAR_HEIGHT/2) -
__betButton_height/2
__betButton_x = PLAYER_INFO_BAR_POSITION[2][0] #initial pos
__gap = (PLAYER_INFO_BAR_POSITION[1][0] - (PLAYER_INFO_BAR_POSITION[2][0] +
PLAYER_INFO_BAR_WIDTH) - __betButton_width*3)/4
__raise_pos   = PLAYER_INFO_BAR_POSITION[2][0] + PLAYER_INFO_BAR_WIDTH +
(PLAYER_INFO_BAR_POSITION[1][0] - (PLAYER_INFO_BAR_POSITION[2][0] +
PLAYER_INFO_BAR_WIDTH) - __betButton_width)/2

gamePageButtons = [Button(kw.CHECK    ,pygame.Rect(__betButton_x - (__gap +
__betButton_width)*1,__betButton_y,__betButton_width,__betButton_height),BUTTON_COLOR,B
UTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH_COLOR),
                   Button(kw.CALL     ,pygame.Rect(__betButton_x - (__gap +
__betButton_width)*2,__betButton_y,__betButton_width,__betButton_height),BUTTON_COLOR,B
UTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH_COLOR),
                   Button(kw.FOLD     ,pygame.Rect(__betButton_x - (__gap +
__betButton_width)*3,__betButton_y,__betButton_width,__betButton_height),BUTTON_COLOR,B
UTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH_COLOR),
                   Button(kw.BET_RAISE,pygame.Rect(__raise_pos ,__betButton_y,__betButt
on_width,__betButton_height)                                    ,BUTTON_COLOR,BUTTON_TOUCH
_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH_COLOR),
```

```
                  Button(kw.DECREASE ,pygame.Rect(__raise_pos - 63                -
__gap,__betButton_y + (__betButton_height -
52.5)/2,63,52.5)   ,BUTTON_COLOR,BUTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_T
OUCH_COLOR,flag=1),
                  Button(kw.INCREASE ,pygame.Rect(__raise_pos + __betButton_width +
__gap,__betButton_y + (__betButton_height -
52.5)/2,63,52.5)   ,BUTTON_COLOR,BUTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_T
OUCH_COLOR,flag=1)]

# login button
loginPageButtons = [Button(kw.SIGN_IN ,pygame.Rect((SCREEN_WIDTH-120)/2
+80,500,120,60),BUTTON_COLOR,BUTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH
_COLOR,flag=-1),
                   Button(kw.SIGN_UP ,pygame.Rect((SCREEN_WIDTH-120)/2 -
80,500,120,60),BUTTON_COLOR,BUTTON_TOUCH_COLOR,BUTTON_ACTIVE_COLOR,BUTTON_ACTIVE_TOUCH_
COLOR,flag=-1),]
```

Evaluator.py:

```
import itertools

class Card:
    """算法來自 https://suffe.cool/poker/evaluator.html """
    # the basics
    STR_RANKS: str = ('2','3','4','5','6','7','8','9','10','11','12','13','14')
    INT_RANKS: range = range(13)
    PRIMES: list[int] = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]

    # conversion from string => int
    CHAR_RANK_TO_INT_RANK: dict[str, int] = dict(zip(list(STR_RANKS), INT_RANKS))
    CHAR_SUIT_TO_INT_SUIT: dict[str, int] = {
        '1': 1,  # spades
        '2': 2,  # hearts
        '3': 4,  # diamonds
        '4': 8,  # clubs
    }
    INT_SUIT_TO_CHAR_SUIT: str = 'x12x3xxx4'  #按八位運算

    @staticmethod
    def new(string: str) -> int:
        rank_int = Card.CHAR_RANK_TO_INT_RANK[string.split('_')[0]]
        suit_int = Card.CHAR_SUIT_TO_INT_SUIT[string.split('_')[1]]
        rank_prime = Card.PRIMES[rank_int]

        bitrank = 1 << rank_int << 16
        suit = suit_int << 12
        rank = rank_int << 8
```

```
        return bitrank | suit | rank | rank_prime

    @staticmethod
    def int_to_str(card_int: int) -> str:
        rank_int = Card.get_rank_int(card_int)
        suit_int = Card.get_suit_int(card_int)
        return f'{Card.STR_RANKS[rank_int]}_{Card.INT_SUIT_TO_CHAR_SUIT[suit_int]}'

    @staticmethod
    def get_rank_int(card_int: int) -> int:
        return (card_int >> 8) & 0xF

    @staticmethod
    def get_suit_int(card_int: int) -> int:
        return (card_int >> 12) & 0xF

    @staticmethod
    def get_bitrank_int(card_int: int) -> int:
        return (card_int >> 16) & 0x1FFF

    @staticmethod
    def get_prime(card_int: int) -> int:
        return card_int & 0x3F

    @staticmethod
    def hand_to_binary(card_strs: list[str]) -> list[int]:
        """
        Expects a list of cards as strings and returns a list
        of integers of same length corresponding to those strings.
        """
        bhand = []
        for c in card_strs:
            bhand.append(Card.new(c))
        return bhand

    @staticmethod
    def prime_product_from_hand(card_ints: list[int]) -> int:
        product = 1
        for c in card_ints:
            product *= (c & 0xFF)
        return product

    @staticmethod
    def prime_product_from_rankbits(rankbits: int) -> int:
        product = 1
        for i in Card.INT_RANKS:
            # if the ith bit is set
```

```
            if rankbits & (1 << i):
                product *= Card.PRIMES[i]

        return product


class LookupTable:
    """
    Number of Distinct Hand Values:

    Straight Flush   10
    Four of a Kind   156      [(13 choose 2) * (2 choose 1)]
    Full Houses      156      [(13 choose 2) * (2 choose 1)]
    Flush            1277     [(13 choose 5) - 10 straight flushes]
    Straight         10
    Three of a Kind  858      [(13 choose 3) * (3 choose 1)]
    Two Pair         858      [(13 choose 3) * (3 choose 2)]
    One Pair         2860     [(13 choose 4) * (4 choose 1)]
    High Card      + 1277     [(13 choose 5) - 10 straights]
    -------------------------
    TOTAL            7462

    Here we create a lookup table which maps:
        5 card hand's unique prime product => rank in range [1, 7462]

    Examples:
    * Royal flush (best hand possible)          => 1
    * 7-5-4-3-2 unsuited (worst hand possible)  => 7462
    """
    MAX_ROYAL_FLUSH: int     = 1
    MAX_STRAIGHT_FLUSH: int  = 10
    MAX_FOUR_OF_A_KIND: int  = 166
    MAX_FULL_HOUSE: int      = 322
    MAX_FLUSH: int           = 1599
    MAX_STRAIGHT: int        = 1609
    MAX_THREE_OF_A_KIND: int = 2467
    MAX_TWO_PAIR: int        = 3325
    MAX_PAIR: int            = 6185
    MAX_HIGH_CARD: int       = 7462

    MAX_TO_RANK_CLASS: dict[int, int] = {
        MAX_ROYAL_FLUSH: 0,
        MAX_STRAIGHT_FLUSH: 1,
        MAX_FOUR_OF_A_KIND: 2,
        MAX_FULL_HOUSE: 3,
        MAX_FLUSH: 4,
        MAX_STRAIGHT: 5,
```

```
        MAX_THREE_OF_A_KIND: 6,
        MAX_TWO_PAIR: 7,
        MAX_PAIR: 8,
        MAX_HIGH_CARD: 9
    }

    RANK_CLASS_TO_STRING: dict[int, str] = {
        0: "Royal Flush",
        1: "Straight Flush",
        2: "Four of a Kind",
        3: "Full House",
        4: "Flush",
        5: "Straight",
        6: "Three of a Kind",
        7: "Two Pair",
        8: "One Pair",
        9: "High Card"
    }

    def __init__(self) -> None:
        """
        Calculates lookup tables
        """
        # create dictionaries
        self.flush_lookup: dict[int, int] = {}
        self.unsuited_lookup: dict[int, int] = {}

        # create the lookup table in piecewise fashion
        # this will call straights and high cards method,
        # we reuse some of the bit sequences
        # import time
        # a = time.time()
        self.flushes()
        self.multiples()
        # print((time.time()-a)*1000)

    def flushes(self) -> None:
        """
        Straight flushes and flushes.

        Lookup is done on 13 bit integer (2^13 > 7462):
        xxxbbbbb bbbbbbbb => integer hand index
        """
        # straight flushes in rank order
        straight_flushes = [
            7936,  # int('0b1111100000000', 2), # royal flush
            3968,  # int('0b111110000000', 2),
```

```
            1984,  # int('0b11111000000', 2),
            992,   # int('0b1111100000', 2),
            496,   # int('0b111110000', 2),
            248,   # int('0b11111000', 2),
            124,   # int('0b1111100', 2),
            62,    # int('0b111110', 2),
            31,    # int('0b11111', 2),
            4111   # int('0b1000000001111', 2) # 5 high
        ]

        # now we'll dynamically generate all the other
        # flushes (including straight flushes)
        flushes = []
        gen = self.get_lexographically_next_bit_sequence(int('0b11111', 2))

        # 1277 = number of high cards
        # 1277 + len(str_flushes) is number of hands with all cards unique rank
        for i in range(1277 + len(straight_flushes) - 1):   # we also iterate over SFs
            # pull the next flush pattern from our generator
            f = next(gen)

            # if this flush matches perfectly any
            # straight flush, do not add it
            notSF = True
            for sf in straight_flushes:
                # if f XOR sf == 0, then bit pattern
                # is same, and we should not add
                if not f ^ sf:
                    notSF = False

            if notSF:
                flushes.append(f)

        # we started from the lowest straight pattern, now we want to start ranking
from
        # the most powerful hands, so we reverse
        flushes.reverse()

        # now add to the lookup map:
        # start with straight flushes and the rank of 1
        # since it is the best hand in poker
        # rank 1 = Royal Flush!
        rank = 1
        for sf in straight_flushes:
            prime_product = Card.prime_product_from_rankbits(sf)
            self.flush_lookup[prime_product] = rank
            rank += 1
```

```
        # we start the counting for flushes on max full house, which
        # is the worst rank that a full house can have (2,2,2,3,3)
        rank = LookupTable.MAX_FULL_HOUSE + 1
        for f in flushes:
            prime_product = Card.prime_product_from_rankbits(f)
            self.flush_lookup[prime_product] = rank
            rank += 1

        # we can reuse these bit sequences for straights
        # and high cards since they are inherently related
        # and differ only by context
        self.straight_and_highcards(straight_flushes, flushes)

    def straight_and_highcards(self, straights: list[int], highcards: list[int]) ->
None:
        """
        Unique five card sets. Straights and highcards.

        Reuses bit sequences from flush calculations.
        """
        rank = LookupTable.MAX_FLUSH + 1

        for s in straights:
            prime_product = Card.prime_product_from_rankbits(s)
            self.unsuited_lookup[prime_product] = rank
            rank += 1

        rank = LookupTable.MAX_PAIR + 1
        for h in highcards:
            prime_product = Card.prime_product_from_rankbits(h)
            self.unsuited_lookup[prime_product] = rank
            rank += 1

    def multiples(self) -> None:
        """
        Pair, Two Pair, Three of a Kind, Full House, and four of a Kind.
        """
        backwards_ranks = list(range(len(Card.INT_RANKS) - 1, -1, -1))

        # 1) Four of a Kind
        rank = LookupTable.MAX_STRAIGHT_FLUSH + 1

        # for each choice of a set of four rank
        for i in backwards_ranks:

            # and for each possible kicker rank
```

```
            kickers = backwards_ranks[:]
            kickers.remove(i)
            for k in kickers:
                product = Card.PRIMES[i]**4 * Card.PRIMES[k]
                self.unsuited_lookup[product] = rank
                rank += 1

        # 2) Full House
        rank = LookupTable.MAX_FOUR_OF_A_KIND + 1

        # for each three of a kind
        for i in backwards_ranks:

            # and for each choice of pair rank
            pairranks = backwards_ranks[:]
            pairranks.remove(i)
            for pr in pairranks:
                product = Card.PRIMES[i]**3 * Card.PRIMES[pr]**2
                self.unsuited_lookup[product] = rank
                rank += 1

        # 3) Three of a Kind
        rank = LookupTable.MAX_STRAIGHT + 1

        # pick three of one rank
        for r in backwards_ranks:

            kickers = backwards_ranks[:]
            kickers.remove(r)
            gen = itertools.combinations(kickers, 2)

            for kickers_2combo in gen:

                c1, c2 = kickers_2combo
                product = Card.PRIMES[r]**3 * Card.PRIMES[c1] * Card.PRIMES[c2]
                self.unsuited_lookup[product] = rank
                rank += 1

        # 4) Two Pair
        rank = LookupTable.MAX_THREE_OF_A_KIND + 1

        tpgen = itertools.combinations(tuple(backwards_ranks), 2)
        for tp in tpgen:

            pair1, pair2 = tp
            kickers = backwards_ranks[:]
            kickers.remove(pair1)
```

```
            kickers.remove(pair2)
            for kicker in kickers:

                product = Card.PRIMES[pair1]**2 * Card.PRIMES[pair2]**2 *
Card.PRIMES[kicker]
                self.unsuited_lookup[product] = rank
                rank += 1

        # 5) Pair
        rank = LookupTable.MAX_TWO_PAIR + 1

        # choose a pair
        for pairrank in backwards_ranks:

            kickers = backwards_ranks[:]
            kickers.remove(pairrank)
            kgen = itertools.combinations(tuple(kickers), 3)

            for kickers_3combo in kgen:

                k1, k2, k3 = kickers_3combo
                product = Card.PRIMES[pairrank]**2 * Card.PRIMES[k1] \
                    * Card.PRIMES[k2] * Card.PRIMES[k3]
                self.unsuited_lookup[product] = rank
                rank += 1

    def get_lexographically_next_bit_sequence(self, bits: int):
        """
        Bit hack from here:
        http://www-graphics.stanford.edu/~seander/bithacks.html#NextBitPermutation

        Generator even does this in poker order rank
        so no need to sort when done! Perfect.
        """
        t = int((bits | (bits - 1))) + 1
        next = t | ((int(((t & -t) / (bits & -bits))) >> 1) - 1)
        yield next
        while True:
            t = (next | (next - 1)) + 1
            next = t | ((((t & -t) // (next & -next)) >> 1) - 1)
            yield next

class Evaluator:

    HAND_LENGTH = 2
    BOARD_LENGTH = 5
```

```
    def __init__(self) -> None:

        self.table = LookupTable()
        self.hand_size_map = {
            2: self._two,
            5: self._five,
            6: self._six,
            7: self._seven
        }

    def evaluate(self, hand: list[int], board: list[int]) -> int:
        """
        This is the function that the user calls to get a hand rank.

        No input validation because that's cycles!
        """
        all_cards = hand + board
        return self.hand_size_map[len(all_cards)](all_cards)

    def _two(self, cards: list[int]) -> int:
        """新添加"""
        if (Card.get_prime(cards[0])) == (Card.get_prime(cards[1])):
            return LookupTable.MAX_PAIR -
(Card.get_prime(cards[0]))*(Card.get_prime(cards[1]))
        else:
            return LookupTable.MAX_HIGH_CARD - max(cards[0] & 0xff,cards[1] & 0xff)

    def _five(self, cards: list[int]) -> int:
        """
        Performs an evalution given cards in integer form, mapping them to
        a rank in the range [1, 7462], with lower ranks being more powerful.

        Variant of Cactus Kev's 5 card evaluator, though I saved a lot of memory
        space using a hash table and condensing some of the calculations.
        """
        # if flush
        if cards[0] & cards[1] & cards[2] & cards[3] & cards[4] & 0xF000:
            handOR = (cards[0] | cards[1] | cards[2] | cards[3] | cards[4]) >> 16
            prime = Card.prime_product_from_rankbits(handOR)
            return self.table.flush_lookup[prime]

        # otherwise
        else:
            prime = Card.prime_product_from_hand(cards)
            return self.table.unsuited_lookup[prime]

    def _six(self, cards: list[int]) -> int:
```

```
        """
        Performs five_card_eval() on all (6 choose 5) = 6 subsets
        of 5 cards in the set of 6 to determine the best ranking,
        and returns this ranking.
        """
        minimum = LookupTable.MAX_HIGH_CARD

        for combo in itertools.combinations(cards, 5):

            score = self._five(combo)
            if score < minimum:
                minimum = score

        return minimum

    def _seven(self, cards: list[int]) -> int:
        """
        Performs five_card_eval() on all (7 choose 5) = 21 subsets
        of 5 cards in the set of 7 to determine the best ranking,
        and returns this ranking.
        """
        minimum = LookupTable.MAX_HIGH_CARD

        for combo in itertools.combinations(cards, 5):
            score = self._five(combo)
            if score < minimum:
                minimum = score

        return minimum

    def get_rank_string(self, hr: int) -> int:
        """
        Returns the class name of hand
        returned from evaluate.
        """
        if hr >= 0 and hr <= LookupTable.MAX_ROYAL_FLUSH:
            class_int = LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_ROYAL_FLUSH]
        elif hr <= LookupTable.MAX_STRAIGHT_FLUSH:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_STRAIGHT_FLUSH]
        elif hr <= LookupTable.MAX_FOUR_OF_A_KIND:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_FOUR_OF_A_KIND]
        elif hr <= LookupTable.MAX_FULL_HOUSE:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_FULL_HOUSE]
        elif hr <= LookupTable.MAX_FLUSH:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_FLUSH]
        elif hr <= LookupTable.MAX_STRAIGHT:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_STRAIGHT]
```

```
        elif hr <= LookupTable.MAX_THREE_OF_A_KIND:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_THREE_OF_A_KIND]
        elif hr <= LookupTable.MAX_TWO_PAIR:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_TWO_PAIR]
        elif hr <= LookupTable.MAX_PAIR:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_PAIR]
        elif hr <= LookupTable.MAX_HIGH_CARD:
            class_int =  LookupTable.MAX_TO_RANK_CLASS[LookupTable.MAX_HIGH_CARD]
        else:
            raise Exception("Inavlid hand rank, cannot return rank class")
        return LookupTable.RANK_CLASS_TO_STRING[class_int]

evaluator = Evaluator()
```

holdem.py:

```
from base import *
from queue import PriorityQueue

class Holdme(BaseGame):
    """ # only calculates\n
        程序入口為 holdem(),調用后運行一回合而不是一整局,因此要運行一整局需配合循環(一局 5 回合)
    """

    def __init__(self,
                 ante:int = 100,
                 pot:int = 0,
                 handList:list[list] = [],
                 communityCardsList:list[str] = [""] * 5,
                 gameRound = 0,
                 small_blind = 0):
        #finish: someone win the game
        super().__init__(total_player=5)
        self.ante= ante
        self.pot = pot
        self.handList = handList
        self.communityCardsList = communityCardsList[:5] + [''] * (5 - len(communityCardsList))
        self.gameRound = gameRound
        self.small_blind = small_blind
        self.__big_blind = self.small_blind + 1
        self.__foldList  = [False] *5
        self.__pokerList = [f'{i}_{j}' for i in range(2,15) for j in range(1,5)]
        self.winnerList = []
        self.__sidePot = []
        self.total_bet = 0
```

```
    def check_game(self):
        """check the blind seat or someone win the game,"""
        for k,v in kw.CASINO_LEVEL.items():
            if self.playersList[2].chip >= v:
                self.ante = round(v * 0.1)
                break
        # ensure blind seat not exceed the range
        self.small_blind = (self.small_blind) % 5
        self.__big_blind = (self.small_blind + 1) % 5

    def game_loop(self,choiceFunc,
                  updateChipFunc = None,
                  updateCardFunc = None,
                  updatePotFunc  = None,
                  ThinkingFunc   = None):
        '''choiceFunc (necessary): real player operator\n
           (OPTIONAL):\n
           updateChipFunc: update each player's chip\n
           updateCardFunc: update card layer if player fold\n
           updatePotFunc: update pot after player bet\n
           ThinkingFunc: sleep the programe for fake thinking of bot\n'''
        self.betting_round(choiceFunc,updateChipFunc,updateCardFunc,updatePotFunc,Think
ingFunc)
        # only 1 player
        if self.winnerList:
            return
        # new round
        self.gameRound += 1

        match self.gameRound:
            case 1:
                self.deal_player()
            case 2:
                self.deal_community(range(0,3))
            case 3:
                self.deal_community(range(3,4))
            case 4:
                self.deal_community(range(4,5))
            case 5:
                self.check_winner()
                return
        # get combo
        for player in self.playersList:
            player.combination(list(filter(lambda x:x != "",self.communityCardsList)))
            player.total_bet = 0

    def betting_round(self,choiceFunc,
```

```
                    updateChipFunc = None,
                    updateCardFunc = None,
                    updatePotFunc  = None,
                    ThinkingFunc   = None):
        current = self.small_blind
        seat_range = range(current,current + len(self.playersList))
        least_bet = 0
        again = 1
        while again > 0:
            for seat in seat_range:
                # find the next player index
                current = (current + 1) % len(self.playersList)

                # avoid index error(small blind)
                if  seat >= len(self.playersList):
                    seat -= len(self.playersList)

                # blind bet
                is_ante = self.total_bet < 2

                if self.gameRound == 0:
                    if    seat == self.small_blind:  least_bet = round(self.ante/2)
                    elif  seat == self.__big_blind:  least_bet = self.ante
                    else: least_bet = 0

                if not self.playersList[seat].fold and not
self.playersList[seat].allin:
                    ### bot thinking
                    if ThinkingFunc and seat != 2:
ThinkingFunc(seat,random.randint(1,7))

                    # player operation
                    result =
self.playersList[seat].decision(is_ante,least_bet,self.ante,choiceFunc)
                    self.total_bet += 1
                    self.pot += result['bet']
                    if result['choice'] == kw.FOLD: self.__foldList[seat] =
self.playersList[seat].fold

                    ### update in interface
                    if updateChipFunc:
updateChipFunc(seat,self.playersList[seat].username,self.playersList[seat].chip)
                    if updatePotFunc :
updatePotFunc(self.pot,seat,result['choice'],result['bet'])
                    if updateCardFunc and self.playersList[seat].fold:
updateCardFunc(seat,self.playersList[seat].hand,self.playersList[seat].fold)
```

```
                    # only one player
                    if self.__foldList.count(False) == 1:
                        self.check_winner()
                        return

                    # add one round if someone raise
                    if result['choice'] == kw.BET_RAISE:
                        least_bet = self.playersList[seat].total_bet
                        seat_range = range(current,current + len(self.playersList) - 1)
                        again += 1
                        break
            again -= 1

    def check_winner(self):
        nonFoldList = PriorityQueue()
        # 獲得未棄牌的玩家信息
        for player in self.playersList:
            if not player.fold:
nonFoldList.put((player.combo[0],player,player.combo[1]))
        # 獲勝玩家
        self.winnerList.append(nonFoldList.get())
        while not nonFoldList.empty():
            player = nonFoldList.get()
            if player[0] != self.winnerList[-1][0]:
                break
            else:
                self.winnerList.append(player)
        for player in self.winnerList:
            player[1].chip += self.pot//len(self.winnerList)

    def deal_player(self):
        '''deal hand to self.handList'''
        # already set hand
        if len(self.handList) == 5:
            for i in range(len(self.handList)):
                self.playersList[i].hand = self.handList[i]
            return None
        # deal card to Player()
        self.handList = []
        for i in range(len(self.playersList)):
            # get 2 hand
            element = random.sample(self.__pokerList,2)
            self.handList.append(element)
            self.playersList[i].hand = element
            for i in range(2):
                self.__pokerList.remove(element[i])
```

```
    def deal_community(self,card_range:range):
        # deal community
        for i in card_range:
            if not self.communityCardsList[i]:
                element = random.choice(self.__pokerList)
                self.communityCardsList[i] = element
                self.__pokerList.remove(element)

if __name__ == '__main__':
    pass
```

input_box.py:

```
from setting import *

class Input:
    """draw a input box in a Surface"""
    def __init__(self,
                 text:str,
                 limitation:int,
                 rect:pygame.Rect,
                 color:tuple,
                 border_color:tuple,
                 active_color:tuple = (0,0,0,0),
                 active_border_color:tuple = (0,0,0,0)
                 ) -> None:
        """limitation: the longest length of input content"""
        self.text = text
        self.limitation = limitation
        self.rect = rect
        self.color= color
        self.border_color = border_color
        self.active_color = active_color
        self.active_border_color = active_border_color
        self.content = ''

    def content_init(self):
        self.content = ''

    def check(self,event:pygame.event):
        # click the input box
        typing = True
        if self.rect.collidepoint(pygame.mouse.get_pos()) and event.type ==
pygame.MOUSEBUTTONDOWN:
            self.border_color,self.active_border_color =
self.active_border_color,self.border_color
            self.color,self.active_color = self.active_color,self.color
            while typing:
```

```
                for event in pygame.event.get():
                    if event.type == pygame.QUIT:
                        raise QuitGame
                    # quit input box,enter and click again
                    elif event.type == pygame.MOUSEBUTTONDOWN or (event.type ==
pygame.KEYDOWN and event.key in (pygame.K_RETURN, pygame.K_KP_ENTER)):
                        self.border_color,self.active_border_color =
self.active_border_color,self.border_color
                        self.color,self.active_color = self.active_color,self.color
                        typing = False
                    # typing
                    elif event.type == pygame.KEYDOWN:
                        if event.key == pygame.K_TAB:
                            pass
                        elif event.key in [pygame.K_BACKSPACE,pygame.K_DELETE]:
                            self.content = self.content[:-1]
                        else:
                            # check text limitation
                            if len(self.content) < self.limitation:
                                self.content += event.unicode
                self.draw()

    def draw(self):
        # reminder
        if len(self.content) == 0:
            text = INPUT_FONT.render(self.text,True,INPUT_FONT_COLOR)
        else:
            text = INPUT_FONT.render(self.content,True,INPUT_FONT_COLOR)
        # render text and input box
        box = pygame.Surface((self.rect.width,self.rect.height),pygame.SRCALPHA)
        pygame.draw.rect(box,self.color,box.get_rect(),0,25)
        box.blit(text,((self.rect.width-text.get_width())/2,(self.rect.height-
text.get_height())/2))
        pygame.draw.rect(box,self.border_color,box.get_rect(),6,25)
        screen.blit(box,(self.rect.x,self.rect.y))
        pygame.display.flip()

loginPageInputs =
[Input(kw.USERNAME  ,20,USERNAME_BOX_RECT  ,TABLE_COLOR           ,COMBO_REMINDER_BG_C
OLOR,COMBO_REMINDER_BG_COLOR,(244, 244, 244)),
                  Input(kw.PASSWORD  ,20,PASSWORD_BOX_RECT  ,COMBO_REMINDER_BG_COLOR,C
OMBO_REMINDER_BG_COLOR,TABLE_COLOR           ,(244, 244, 244)),
                  Input(kw.C_PASSWORD,20,C_PASSWORD_BOX_RECT,COMBO_REMINDER_BG_COLOR,C
OMBO_REMINDER_BG_COLOR,TABLE_COLOR           ,(244, 244, 244))]
```

keywords.py:

```
# title
TITLE_TEXT = "WELCOME to Hold'em"

# button
ALL_IN = 'all in'
CHECK = 'check'
BET_RAISE = 'raise'
FOLD  = 'fold'
CALL = 'call'
INCREASE = '+'
DECREASE = '-'

# string
C_PASSWORD = 'confirm password'
SIGN_IN = 'sign in'
SIGN_UP = 'sign up'

#login info
SIGN_IN_INFO = 'Please login'
SIGN_IN_SUCCESSE = 'sign in successful'
SIGN_IN_FAIL = 'User or password invalid'
SIGN_UP_INFO = "sign up (Password must be 8 characters long with numbers and letters)"
SIGN_UP_SUCCESSE = 'sign up successful'
SIGN_UP_FAIL = 'User exist or password invalid'

# ===================================NO
CHANGE=========================================
# json key,no space
PLAYER_DATA = 'player_data'
TYPE = 'type'
USERNAME = 'username'
PASSWORD = 'password'
CHIP = 'chip'

CASINO_LEVEL = {
    'president':100000000,
    'rich'     :1000000,
    'poor'     :10000,
    'bankrupt' :100,
}
```

login.py:

```
import json
import os
import keywords as kw

path = os.path.dirname(__file__).replace('code','data') + '\\player.json'

def load_players(path:str = path) -> list[dict]:
    '''return a list include player data [{username:...,},{...}...]'''
    with open(path,mode='r',encoding='utf-8') as fp:
        return json.load(fp)[kw.PLAYER_DATA]

def sign_in(username:str,password:str):
    """return player info after login successful,\n
       else return False"""
    for player in load_players():
        # 拒绝机器人登录
        if player[kw.USERNAME] == username and player[kw.PASSWORD] == password and
player[kw.TYPE]:
            del player[kw.PASSWORD]
            return player
    return False

def sign_up(username:str,password:str,c_password:str,real_player:bool = True) -> bool:
    def check_password(password:str):
        letter = False
        digit = False
        if len(password) >= 8:
            for char in password:
                if char.isalpha():
                    letter = True
                if char.isdigit():
                    digit = True
                if letter and digit:
                    break
        return letter and digit
    content = load_players()
    # username exist
    for user in content:
        if user[kw.USERNAME] == username:
            return False
    # invalid password
    if not check_password(password) or password != c_password:
        return False
    content.append({
        kw.TYPE: int(real_player),  #type 1 is real player,0 is bot
```

```
        kw.USERNAME:username,
        kw.PASSWORD:password,
        kw.CHIP: 500000
    })
    with open(path,mode='w',encoding='utf-8') as fp:
        json.dump({kw.PLAYER_DATA:content},fp,indent=4)
    return True

def get_testing_data() -> dict:
    '''testing user data(non save)'''
    return {
        kw.TYPE: 1,  #type 1 is real player,0 is bot
        kw.USERNAME:'test_username',
        kw.CHIP: 1000000
    }

def get_bot() -> list[dict]:
    '''all bot info but without password'''
    botList = []
    for bot in load_players():
        del bot[kw.PASSWORD]
        if bot[kw.TYPE] == 0:
            botList.append(bot)
    return botList

def save_game(players_info:list[dict] = []):
    """save the player data to json"""
    ori_players_info = load_players()
    for ori_player in ori_players_info:
        for player in players_info:
            if ori_player[kw.USERNAME] == player[kw.USERNAME]:
                ori_player[kw.CHIP] = player[kw.CHIP]
    with open(path,mode='w',encoding='utf-8') as fp:
        json.dump({kw.PLAYER_DATA:ori_players_info},fp,indent=4)

def calculate_ranking(ori_data:dict) -> list[list]:
    sorted_data = sorted(ori_data, key=lambda x: x[kw.CHIP], reverse=True)
    ranked_data = []
    current_rank = 0
    previous_chip = None
    for item in sorted_data:
        if previous_chip is None or item[kw.CHIP] != previous_chip:
            current_rank += 1
        ranked_data.append([current_rank,item[kw.USERNAME], item[kw.CHIP]])
        previous_chip = item[kw.CHIP]
    return ranked_data
```

```
if __name__ == '__main__':
    pass
```

main.py:

```
from rendering import *
from holdem import *
from login import *

def login_page(login = False) -> dict:
    '''return real player info after login '''
    # non login
    if login: return get_testing_data()
    # login
    invalid = kw.C_PASSWORD
    attempts = 0
    temp_text = ''
    reminder_text = kw.SIGN_IN_INFO
    while not login:
        attempts += 1
        screen.fill(SCREEN_COLOR)
        draw_ranking(calculate_ranking(load_players()))
        draw_table(0)
        draw_introduction(attempts == 1)
        reminder = render_reminder(reminder_text,LOGIN_REMINDER_BG_COLOR,20)
        screen.blit(reminder,((SCREEN_WIDTH-
reminder.get_width())/2,SCREEN_HEIGHT*250/SCREEN_HEIGHT))
        # recieve input
        result = interact(buttonList =
loginPageButtons,inputList=loginPageInputs,invalidList=[invalid])
        # sign in
        if result['choice'] == kw.SIGN_IN:  # sign in
            reminder_text = kw.SIGN_IN_INFO
            if invalid == '':               # no sign in when Sign up
                invalid = kw.C_PASSWORD
                continue                    # skip to login
            login = sign_in(result[kw.USERNAME],result[kw.PASSWORD])
            temp_text = kw.SIGN_IN_SUCCESSE if login else kw.SIGN_IN_FAIL
            if invalid:
                draw_temp_reminder(temp_text,[SCREEN_WIDTH/2,(SCREEN_HEIGHT*350/SCREEN_
HEIGHT)])
        # sign up
        elif result['choice'] == kw.SIGN_UP:# sign up
            reminder_text = kw.SIGN_UP_INFO
            temp_text = kw.SIGN_UP_SUCCESSE if
sign_up(result[kw.USERNAME],result[kw.PASSWORD],result[kw.C_PASSWORD]) else
kw.SIGN_UP_FAIL
```

```
            if not invalid:
                draw_temp_reminder(temp_text,[SCREEN_WIDTH/2,(SCREEN_HEIGHT*350/SCREEN_
HEIGHT)])
            invalid = ''
        for input_box in loginPageInputs: input_box.content_init()
    return login

def holdem_page(game:Holdme):
    screen.fill(SCREEN_COLOR)
    game.check_game()
    while True:
        clock.tick(FPS)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
        draw_table()
        draw_pot(game.pot)
        # draw_blind()
        # draw every player
        for seat in range(5):
draw_players(seat,game.playersList[seat].username,game.playersList[seat].chip)
        r = game.gameRound
        if r >= 1:
            for seat in range(5):
                draw_hand(seat,game.playersList[seat].hand,game.playersList[seat].fold,
(r == 1 and not game.winnerList),game.winnerList)
        draw_combo(game.get_players_info('combo'))
        # end game
        if r >= 2:
            draw_community(game.communityCardsList,range(0,3),r == 2 and not
game.winnerList)
        if r >= 3:
            draw_community(game.communityCardsList,range(3,4),r == 3 and not
game.winnerList)
        if r >= 4:
            draw_community(game.communityCardsList,range(4,5),r == 4 and not
game.winnerList)
        if game.winnerList:
            draw_winner(game.winnerList)
            return True
        game.game_loop(interact,
                       draw_players,
                       draw_hand,
                       draw_betting,
                       draw_consideration)

def other_game_page():...
```

```
def main():
    try:
        player_info = login_page(login=False)  # login = true: non login
        samll_blind = 0
        while True:
            # game = choose_game()
            # if game == 'holdem':
            game = Holdme(small_blind=samll_blind)
            game.init_player(get_bot(),player_info)
            if holdem_page(game):
                player_info = game.save_game()[2]
                save_game(game.save_game())
            samll_blind += 1
    except QuitGame:
        if 'game' in locals():
            save_game(game.save_game())
        pygame.quit()
        quit()
    except Exception:
        main()

if __name__ == '__main__':
    main()
```

player.py:

```
import keywords as kw
import random
from python.project.card_game.code.evaluator import Card,evaluator

class Player():

    def __init__(self,
                 username:str,
                 chip:int,
                 hand:list = [],
                 fold = False) -> None:
        self.username = username
        self.chip = chip
        self.hand = hand
        self.fold = fold
        self.allin= False   # ***temp
        self.combo= ['',''] # combo int, combo name
        self.total_bet = 0
        if self.chip < 10000: self.chip = 10000 # ***temp

    def decision(self,is_ante:bool,least_bet:int,least_raise:int,choiceFunc) -> dict:
```

```
        ''' return a dict{choice,bet}'''
        # calculate player's least bet,(not least bet for a round)
        least_bet -= self.total_bet
        #used in gui
        if is_ante:                         # blind seat or allin
            invalidList = [kw.INCREASE,kw.DECREASE,kw.BET_RAISE,kw.FOLD,kw.CHECK]
        elif not self.hand:                 # first round
            invalidList = [kw.INCREASE,kw.DECREASE,kw.BET_RAISE,kw.FOLD,kw.CALL]
        # ================================================================================
***temp
        elif self.chip <= least_bet + least_raise:  # allin
            invalidList = [kw.INCREASE,kw.DECREASE,kw.BET_RAISE,kw.CHECK]
            self.allin = True
        # ================================================================================
        elif least_bet > 0:                 # someone bet
            invalidList = [kw.CHECK]
        elif least_bet == 0 and self.hand:# first seat or allin
            invalidList = [kw.CALL]
        # receive data
        result:dict = choiceFunc(self.chip,least_bet,least_raise,self.fold,invalidList)
        # handle data
        choice = result['choice']
        # ================================================================================
***temp
        if result['choice'] == kw.CALL and self.allin or (self.chip <= least_bet +
least_raise and is_ante):
            self.allin = True
            bet = self.chip
            choice = kw.ALL_IN
        #================================================================================
        elif result['choice'] == kw.CALL or is_ante: bet = least_bet
        elif result['choice'] == kw.BET_RAISE:       bet = result['bet']
        elif result['choice'] == kw.CHECK:           bet = 0
        elif result['choice'] == kw.FOLD:            bet,self.fold = 0,True

        # upload data
        self.total_bet += bet
        self.chip -= bet
        return {'choice':choice,'bet':bet}

    def combination(self,community:list):
        community = list(filter(lambda x: x,community))
        rank_int = evaluator.evaluate([Card.new(card) for card in
self.hand],[Card.new(card) for card in community])
        combo = evaluator.get_rank_string(rank_int)
        self.combo = [rank_int,combo]
```

```
class Bot(Player):
    def __init__(self,
                 username,
                 chip,
                 hand=[],
                 fold = False) -> None:
        super().__init__(username,chip,hand,fold)

    def decision(self,is_ante:bool,least_bet:int,least_raise:int,choiceFunc):
        # filter invalid ones
        weights = [100]
        if is_ante:          # ante
            validList = [kw.CALL]
        elif not self.hand:  # first round
            validList = [kw.CHECK]
        # ================================================================================
***temp
        elif self.chip <= least_bet + least_raise:  # allin
            validList = [kw.FOLD,kw.CALL]
            weights = [80,20]
            self.allin = True
        # ================================================================================
        elif least_bet == 0: # nobody bet
            validList = [kw.FOLD,kw.CHECK,kw.BET_RAISE]
            weights = [5,60,45]
        elif least_bet > 0:  # someone bet
            validList = [kw.FOLD,kw.CALL,kw.BET_RAISE]
            weights = [15,60,25]
        # calculate player's least bet,(not least bet for a round)
        least_bet = least_bet - self.total_bet
        # make decision
        choice = random.choices(validList,weights=weights,k=1)[0]
        # ================================================================================
***temp
        if choice == kw.CALL and self.allin or (self.chip <= least_bet + least_raise
and is_ante):
            self.allin = True
            bet = self.chip
            choice = kw.ALL_IN
        #================================================================================
        elif choice == kw.CALL or is_ante: bet = least_bet
        elif choice == kw.BET_RAISE:       bet = least_bet + least_raise
        elif choice == kw.CHECK:           bet = 0
        elif choice == kw.FOLD:            bet,self.fold = 0,True
        self.chip -= bet
        self.total_bet += bet
        return {'choice':choice,'bet':bet}
```

rendering.py:

```
from setting import *
from button import *
from input_box import *
import keywords as kw
import time

def draw_ranking(ranked_data:list[list]):
    #暫時寫的垃圾
    h = 0
    for item in ranked_data:
        a = render_reminder(f'{item[0]}.{item[1]}',(0,0,0),width = 120)
        b = render_reminder(f'${item[2]}',(0,0,0),width=100)
        h += a.get_height()
        screen.blit(a,(0,h))
        screen.blit(b,(100,h))
        pygame.display.flip()

def draw_introduction(isAnimation:bool = True):
    title = TITLE_FONT.render(kw.TITLE_TEXT,True,TITLE_FONT_COLOR)
    init_pos = ((SCREEN_WIDTH-title.get_width())/2,(SCREEN_HEIGHT-
title.get_height())/2)
    final_position = ((SCREEN_WIDTH-title.get_width())/2,150)
    if isAnimation:
        move_animation(title,init_pos,final_position,1,0)
    screen.blit(title,final_position)

def draw_temp_reminder(text:str,pos:list[int,int],bg_color:tuple =
LOGIN_REMINDER_BG_COLOR,posDeviation:int = 35,time:int = 1):
    reminder = render_reminder(text,bg_color,20)
    pos[0] -= reminder.get_width()/2
    move_animation(reminder,pos,(pos[0],pos[1] - posDeviation),time,alpha_end = 0)

def choose_game():...

def draw_table(num:int = 5):
    '''num: number of poker place'''
    pygame.draw.rect(screen,TABLE_COLOR,TABLE_RECT,0,1000)
    pygame.draw.rect(screen,TABLE_PADDING_COLOR,TABLE_RECT,50,1000)
    pygame.draw.rect(screen,TABLE_BORDER_COLOR,TABLE_RECT,10,1000)
    for i in range(num):
        x = TABLE_RECT.x + (TABLE_WIDTH - ((POKER_PLACE_WIDTH)*num + GAP*(num-1)))/2 +
((POKER_PLACE_WIDTH + GAP) * i)
        y = ((9*(TABLE_RECT.y+TABLE_HEIGHT))+(49*TABLE_RECT.y))/58   # ac:cb = 9:49
        pygame.draw.rect(screen,POKER_PLACE_COLOR,(x,y,POKER_PLACE_WIDTH,POKER_PLACE_HE
IGHT),3,5)
```

```
    pygame.display.flip()

def draw_players(seat:int,playerName:str,playerChip:int):
    """player seat : 0 - 4"""
    pygame.draw.rect(PLAYER_INFO_BAR_LIST[seat],PLAYER_INFO_BAR_COLOR,(0,0,PLAYER_INFO_
BAR_WIDTH,PLAYER_INFO_BAR_HEIGHT),0,20)
    y = PLAYER_INFO_BAR_HEIGHT - PLAYER_ICON.get_height() - 20
    PLAYER_INFO_BAR_LIST[seat].blit(PLAYER_ICON,(0,y))
    PLAYER_INFO_BAR_LIST[seat].blit(PLAYER_NAME_FONT.render(playerName,True,PLAYER_NAME
_FONT_COLOR),(70,y + 5))
    PLAYER_INFO_BAR_LIST[seat].blit(PLAYER_NAME_FONT.render(f'$ {round(playerChip)}',Tr
ue,PLAYER_NAME_FONT_COLOR),(70,y + 35))
    screen.blit(PLAYER_INFO_BAR_LIST[seat],PLAYER_INFO_BAR_POSITION[seat])
    pygame.display.flip()

def render_reminder(text:str,bg_color:tuple[int,int,int],padding:int = 10,width:int =
None) -> pygame.Surface:
    # reminder_height = REMINDER_FONT.render('p',True,REMINDER_FONT_COLOR).get_height()
* 1.3 # take the highest letter
    reminder = REMINDER_FONT.render(text,True,REMINDER_FONT_COLOR)
    if not padding: padding = 0
    if not width: width = reminder.get_width() + padding *2
    reminder_bg = pygame.Surface((width,REMINDER_HEIGHT),pygame.SRCALPHA)
    pygame.draw.rect(reminder_bg,bg_color,reminder_bg.get_rect(),0,12)
    reminder_bg.blit(reminder,((reminder_bg.get_width()-
reminder.get_width())/2,(reminder_bg.get_height()-reminder.get_height())/2))
    return reminder_bg

def render_pot_info(bet:int) -> pygame.Surface:
    chipList = {
        'black' :100000, # black : 100000 =< bet
        'yellow':50000,  # yellow:  50000 =< bet < 100000
        'red'   :10000,  # red   :  10000 =< bet < 50000
        'blue'  :5000,   # blue  :   5000 =< bet < 10000
        'green' :2500,   # green :   2500 =< bet < 5000
        'white' :0       # white :      0 =< bet < 2500
        }
    for color,v in chipList.items():
        if bet >= v:
            break
    reminder =
render_reminder(str(bet),BETTING_REMINDER_BG_COLOR,BETTING_REMINDER_PADDING)
    composition = pygame.Surface((CHIP_WIDTH/1.5 +
reminder.get_width(),max(CHIP_HEIGHT,reminder.get_height())),pygame.SRCALPHA)
    composition.fill(TABLE_COLOR)
    composition.blit(reminder,(CHIP_WIDTH/1.5,(composition.get_height() -
reminder.get_height())/2))
```

```
    composition.blit(CHIP[color],(0,(composition.get_height() -
CHIP[color].get_height())/2))
    return composition

def draw_combo(playerCombination:list):
    '''draw all players combo by using render_reminder()'''
    for player in playerCombination:
        reminder =
render_reminder(player[1],COMBO_REMINDER_BG_COLOR,COMBO_REMINDER_PADDING)
        if player == playerCombination[2] and player[1]:
            position = ((SCREEN_WIDTH -
reminder.get_width())/2,((9*(TABLE_RECT.y+TABLE_HEIGHT))+(49*TABLE_RECT.y))/58 +
POKER_PLACE_HEIGHT + GAP)
            screen.blit(reminder,position)
        elif CHEATING_MODE:...

def draw_pot(pot:int):
    pot_reminder = render_pot_info(pot)
    screen.blit(pot_reminder,((SCREEN_WIDTH-pot_reminder.get_width())/2 - (CHIP_WIDTH -
CHIP_WIDTH/1.5),TABLE_RECT.y + TABLE_HEIGHT/2))

def draw_betting(pot:int,seat:int,choice:str,bet:int):
    draw_pot(pot)
    PLAYER_BETTEING_INFO_LIST[seat].fill(SCREEN_COLOR)
    # text
    text = choice if choice in [kw.FOLD,kw.CHECK] else f'{choice}:{bet}'
    player_reminder =
render_reminder(text,BETTING_REMINDER_BG_COLOR,BETTING_REMINDER_PADDING)
    PLAYER_BETTEING_INFO_LIST[seat].blit(player_reminder,((PLAYER_BETTEING_INFO_LIST[se
at].get_width() - player_reminder.get_width())/2,0))
    move_animation(PLAYER_BETTEING_INFO_LIST[seat],PLAYER_INFO_BAR_POSITION[seat],
                   (PLAYER_INFO_BAR_POSITION[seat][0],PLAYER_INFO_BAR_POSITION[seat][1]
- player_reminder.get_height()),0.4,0)
    if bet: # except fold or check
        bet_reminder = render_reminder(f'+{bet}',BETTING_REMINDER_BG_COLOR,10)
        bet_x = (SCREEN_WIDTH - bet_reminder.get_width())/2
        bet_y = TABLE_RECT.y + TABLE_HEIGHT/2
        bet_frame = screen.copy()
        move_animation(bet_reminder,(bet_x,bet_y),(bet_x,bet_y +
bet_reminder.get_height()),1,-100)
        screen.blit(bet_frame,(0,0))
        pygame.display.flip()
        move_animation(bet_reminder,(bet_x,bet_y +
bet_reminder.get_height()),(bet_x,bet_y),1,alpha_end=-100)

def draw_blind(small_blind = 1):...
```

```
def draw_winner(winnerList:list,waiting_time:int = 20):
    #暫時寫的垃圾
    y = SCREEN_HEIGHT*0.49
    winner_text = render_reminder('winner:',(0,0,0))
    screen.blit(winner_text,((SCREEN_WIDTH-winner_text.get_width())/2,y))
    for winner in winnerList:
        player_info_text = render_reminder(f'{str(winner[1].username)}
({str(winner[2])})', (77, 159, 249))
        y += player_info_text.get_height()
        screen.blit(player_info_text,((SCREEN_WIDTH-player_info_text.get_width())/2,y))
    pygame.display.flip()
    start_time = time.time()
    while True:
        clock.tick(FPS)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                raise QuitGame
        progress = min((time.time() - start_time) / waiting_time,1)
        if progress >= 1:
            break

def draw_hand(seat:int,hand:list,fold = False,deal:bool = False,show:bool = False):
    '''draw all hand card'''
    for i in range(2):
        draw_card(seat,hand[i],HAND_POSITION[seat][i],deal,fold,show)

def draw_community(community:list,deal_range:range,deal:bool = False):
    '''"deal_range" is the range of dealing card(start with 0)\n'''
    for i in deal_range:
        draw_card(-1,community[i],COMMUNITY_CARDS_POSITION[i],deal)

def draw_card(seat:int,card_id:str,position:tuple,deal:bool = False,fold:bool =
False,show:bool = False):
    '''playerId 1-5 == hand, -1 == community cards'''
    if deal:
        move_animation(CARD_BACK,POKER_INITIAL_POSITION,position,0.5)
    #2 player or -1 community
    if seat in [-1,2] or CHEATING_MODE or show:
        screen.blit(POKER[card_id],position)
    #bot
    else:
        screen.blit(CARD_BACK,position)
    #fold
    if fold == True:
        fold_layer = pygame.Surface((POKER_WIDTH,POKER_HEIGHT),pygame.SRCALPHA)
        pygame.draw.rect(fold_layer,(0,0,0,128),fold_layer.get_rect(),0,2)
        screen.blit(fold_layer,position)
```

```
    pygame.display.flip()

def draw_consideration(seat:int,pause_time:int = 1):
    """thinking time is limited for 15s"""
    start_time = time.time()
    max_time = 1
    consider_bar_w = PLAYER_INFO_BAR_WIDTH * 0.8
    consider_bar_h = 8
    x = PLAYER_INFO_BAR_POSITION[seat][0] + (PLAYER_INFO_BAR_WIDTH - consider_bar_w)/2
    y = PLAYER_INFO_BAR_POSITION[seat][1] + (20 - consider_bar_h)/2
    frame = screen.copy()
    r = pygame.Rect(x,y,consider_bar_w,consider_bar_h)
    pygame.draw.rect(screen,(0,0,0),r,0,6)
    while True:
        clock.tick(FPS)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                raise QuitGame
        progress = (time.time() - start_time) / max_time + 0.05
        w = (consider_bar_w) * progress
        pygame.draw.rect(screen,(255,255,255), (x,y,w,consider_bar_h),0,6)
        pygame.display.flip()
        if progress >= 1 or (time.time() - start_time)/pause_time >= 1:
            screen.blit(frame,(0,0))
            break

def move_animation(obj:pygame.Surface,init_pos:tuple,final_pos:tuple,duration:int
                   ,alpha_start:float=255,alpha_end = 255):
    '''animation'''
    frame = screen.copy()
    start_time = time.time()
    alpha = alpha_start
    while True:
        clock.tick(FPS)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                raise QuitGame
        #check animation time
        progress = min((time.time() - start_time) / duration,1)
        #change pos
        x = init_pos[0] + (final_pos[0] - init_pos[0]) * progress
        y = init_pos[1] + (final_pos[1] - init_pos[1]) * progress
        #set alpha channel
        increment = (alpha_end - alpha_start) / (duration * FPS)
        if   (alpha_start < alpha_end and alpha >= alpha_end) or (alpha_start >
alpha_end and alpha <= alpha_end):
            alpha = alpha_end
```

```
        else:
            alpha += increment
        obj.set_alpha(alpha)
        # blit
        screen.blit(frame,(0,0))
        screen.blit(obj,(x,y))
        pygame.display.flip()
        #end the animation
        if progress >= 1 and alpha == alpha_end:
            # console.log(123)
            break

def interact(playerChip:int = -1,
             least_bet:int  = 0,
             least_raise:int= 0,
             press:bool = False,
             invalidList:list = [],
             buttonList:list[Button] = gamePageButtons,
             inputList:list[Input] = []) -> dict:
    '''press: True will pass,return button name\n
       invalidList: receive keyword to ban button'''
    bet = least_raise + least_bet
    result = {}

    def calculate_bet(raise_bet:int):
        nonlocal bet
        bet += raise_bet
        if   bet >= playerChip:  bet = playerChip  # excced max
        elif bet < least_raise + least_bet: bet = least_bet + least_raise   # exceed
min

    while not press:
        clock.tick(FPS)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                raise QuitGame
            # only check
            for input_box in inputList:
                if input_box.text not in invalidList:
                    input_box.check(event)
            for button in buttonList:
                #check button
                if (button.text not in invalidList) and button.check(event):
                    result['choice'] = button.text
                    if   button.flag == 1:
                        calculate_bet(button.adjust_value(least_raise))  #暂时处理，实际
数值于PLayer()中定
```

```
                        continue
                    elif button.flag == 0:
                        result['bet'] = bet
                    elif button.flag == -1:
                        result.update({input_box.text:input_box.content for input_box
in inputList})
                    press = True
        # only draw
        for button in buttonList:
            value = None
            if   button.text == kw.CALL: value = playerChip if least_bet > playerChip
else least_bet
            elif button.text == kw.BET_RAISE: value = bet
            button.draw(button.text in invalidList,value)

        for input_box in inputList:
            if input_box.text not in invalidList: input_box.draw()
    return result
```

setting.py:

```
import os.path
import pygame
import keywords as kw
# ==================================CAN
CHANGE=========================================
# screen setting
SCREEN_HEIGHT= 750
SCREEN_WIDTH = 1380
SCREEN_COLOR = (242, 225, 231)

# FPS setting
FPS = 120

# title
TITLE_FONT_SIZE = 72
TITLE_FONT_COLOR= (255,255,255)

# login page
INPUT_BOX_WIDTH = 320
INPUT_FONT_SIZE = 22
INPUT_FONT_COLOR= (231, 231, 231)
LOGIN_REMINDER_BG_COLOR = (5, 36, 19)

# table
TABLE_WIDTH = 1000
TABLE_HEIGHT= 580
TABLE_COLOR = (17, 113, 61)
```

```
TABLE_PADDING_COLOR= (77, 39, 18)
TABLE_BORDER_COLOR = (0, 0, 0)

# cards
GAP = 5
POKER_WIDTH = 87.5
POKER_HEIGHT= 127.05
POKER_PLACE_COLOR = (204, 204, 204)
POKER_PLACE_WIDTH = 95
POKER_PLACE_HEIGHT= 137.94

# player info
PLAYER_ICON_WIDTH = 60
PLAYER_ICON_HEIGHT= 60
PLAYER_NAME_FONT_SIZE = 20
PLAYER_NAME_FONT_COLOR= (220, 220, 220)
PLAYER_INFO_BAR_WIDTH = 220
PLAYER_INFO_BAR_HEIGHT= 100
PLAYER_INFO_BAR_COLOR = (68, 45, 53)
PLAYER_INFO_BAR_MARGIN= 5   # distance from the edge of scree

# player betting info
PLAYER_BETTEING_INFO_WIDTH = 160
PLAYER_BETTEING_INFO_HEIGHT= 70

# chips
CHIP_WIDTH = 40
CHIP_HEIGHT= 40

# button  edit:(in button.py)
BUTTON_FONT_SIZE = 20
BETTING_SIZE_FONT_SIZE = 15
BUTTON_FONT_COLOR = (255,255,255)
BUTTON_COLOR = (0,0,0)
BUTTON_TOUCH_COLOR  = (68, 45, 53)
BUTTON_ACTIVE_COLOR = (68, 45, 53)
BUTTON_ACTIVE_TOUCH_COLOR = (255,255,255)

# reminder
COMBO_REMINDER_PADDING  = 10
COMBO_REMINDER_BG_COLOR = (7, 61, 31)
BETTING_REMINDER_PADDING  = 25
BETTING_REMINDER_BG_COLOR = (11, 15, 11)
REMINDER_HEIGHT     = 30
REMINDER_FONT_SIZE  = 20
REMINDER_FONT_COLOR = (220, 220, 220)
```

```
#cheat
CHEATING_MODE = False

# ==================================NO
CHANGE=========================================

# quitgame
class QuitGame(Exception):
    pass

def load_image(path,size = None) -> dict:
    """return a dictionary include {file_name:image_surface,...}"""
    # nameList = os.listdir(path+'\\PNG-cards-1.3')
    nameList = [i.replace('.png','') for i in os.listdir(path)]
    imageList = {}
    for i in range(len(nameList)):
        image = pygame.image.load(f'{path}\\{nameList[i]}.png')
        if size:
            image = pygame.transform.smoothscale(image,size)
        imageList[nameList[i]] = image
    return imageList

# path
imagePath = os.path.dirname(__file__).replace('code','image')
fontPath  = os.path.dirname(__file__).replace('code','font')

# load image
PLAYER_ICON =
pygame.transform.smoothscale(pygame.image.load(f'{imagePath}\\User.jpg'),(PLAYER_ICON_W
IDTH,PLAYER_ICON_HEIGHT))
CARD_BACK   =
pygame.transform.smoothscale(pygame.image.load(f'{imagePath}\\card_back.png'),(POKER_WI
DTH,POKER_HEIGHT))
POKER = load_image(imagePath+'\\cards',(POKER_WIDTH,POKER_HEIGHT))
CHIP  = load_image(imagePath+'\\chips',(CHIP_WIDTH,CHIP_HEIGHT))

# init
pygame.init()
pygame.display.set_caption('Card Game')
clock  = pygame.time.Clock()
screen = pygame.display.set_mode((SCREEN_WIDTH,SCREEN_HEIGHT))

# title
TITLE_FONT = pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',TITLE_FONT_SIZE)

# login page
```

```
USERNAME_BOX_RECT    = pygame.Rect((SCREEN_WIDTH-
INPUT_BOX_WIDTH)/2,300,INPUT_BOX_WIDTH,INPUT_FONT_SIZE*2)
PASSWORD_BOX_RECT    = pygame.Rect((SCREEN_WIDTH-
INPUT_BOX_WIDTH)/2,370,INPUT_BOX_WIDTH,INPUT_FONT_SIZE*2)
C_PASSWORD_BOX_RECT = pygame.Rect((SCREEN_WIDTH-
INPUT_BOX_WIDTH)/2,440,INPUT_BOX_WIDTH,INPUT_FONT_SIZE*2)
INPUT_FONT = pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',INPUT_FONT_SIZE)

# button
BUTTON_FONT = pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',BUTTON_FONT_SIZE)
BETTING_SIZE_FONT =
pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',BETTING_SIZE_FONT_SIZE)
buttonList = [...] #(in button.py)

# reminder
REMINDER_FONT = pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',REMINDER_FONT_SIZE)

# table
TABLE_RECT = pygame.Rect((SCREEN_WIDTH-TABLE_WIDTH)/2,(3/14)*(SCREEN_HEIGHT-
TABLE_HEIGHT),TABLE_WIDTH,TABLE_HEIGHT)

# player info
PLAYER_NAME_FONT =
pygame.font.Font(f'{fontPath}\\FreeSansBold.ttf',PLAYER_NAME_FONT_SIZE)
PLAYER_INFO_BAR_LIST =
[pygame.Surface((PLAYER_INFO_BAR_WIDTH,PLAYER_INFO_BAR_HEIGHT),pygame.SRCALPHA) for i
in range(5)]
PLAYER_INFO_BAR_POSITION= [(SCREEN_WIDTH - PLAYER_INFO_BAR_MARGIN -
PLAYER_INFO_BAR_WIDTH,TABLE_RECT.y + 150),
                           (SCREEN_WIDTH - PLAYER_INFO_BAR_MARGIN -
PLAYER_INFO_BAR_WIDTH,TABLE_RECT.y + TABLE_HEIGHT + REMINDER_HEIGHT),
                           (SCREEN_WIDTH/2 - PLAYER_INFO_BAR_WIDTH/2,TABLE_RECT.y +
TABLE_HEIGHT + REMINDER_HEIGHT),
                           (PLAYER_INFO_BAR_MARGIN,TABLE_RECT.y + TABLE_HEIGHT +
REMINDER_HEIGHT),
                           (PLAYER_INFO_BAR_MARGIN,TABLE_RECT.y + 150)]

# player betting info
PLAYER_BETTEING_INFO_LIST =
[pygame.Surface((PLAYER_INFO_BAR_WIDTH,REMINDER_HEIGHT),pygame.SRCALPHA) for i in
range(5)]

# cards
POKER_INITIAL_POSITION = ((SCREEN_WIDTH/2)-(POKER_WIDTH/2),0)
POKER_TABLE_START_POSITION = (SCREEN_WIDTH/2 - (POKER_WIDTH/2) + (-(POKER_PLACE_WIDTH -
POKER_WIDTH) -GAP -(POKER_WIDTH))*2,
```

```
                            ((TABLE_RECT.y) * 96911 + (TABLE_RECT.y + TABLE_HEIGHT)*
19089)/116000) #ac:cb = 19089:96911

HAND_POSITION = [[(PLAYER_INFO_BAR_POSITION[i][0] + ((PLAYER_INFO_BAR_WIDTH -
(POKER_WIDTH*2 - 30))/2),
                  PLAYER_INFO_BAR_POSITION[i][1] - POKER_HEIGHT - REMINDER_HEIGHT),
                 (PLAYER_INFO_BAR_POSITION[i][0] + PLAYER_INFO_BAR_WIDTH -
((PLAYER_INFO_BAR_WIDTH - (POKER_WIDTH*2 - 30))/2)-(POKER_WIDTH),
                  PLAYER_INFO_BAR_POSITION[i][1] - POKER_HEIGHT - REMINDER_HEIGHT)]
                 for i in range(len(PLAYER_INFO_BAR_POSITION))]

COMMUNITY_CARDS_POSITION = [(POKER_TABLE_START_POSITION[0] + ((POKER_PLACE_WIDTH -
POKER_WIDTH) + GAP + (POKER_WIDTH))* i,
                             POKER_TABLE_START_POSITION[1]) for i in range(5)]
```